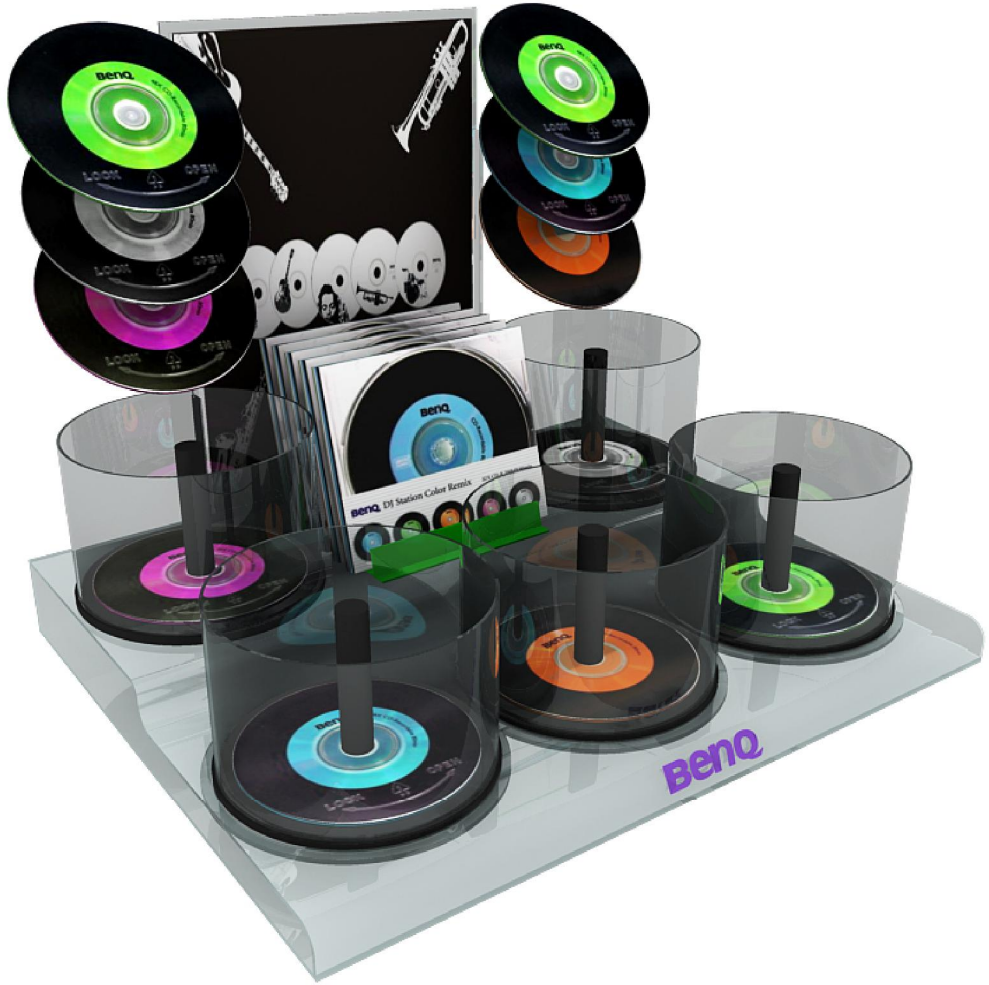

BenQ
BenQ DJ Station Color Remix
BenQ

通常称，栈和队列是限定**插入和删除**只能在表的“**端点**”进行的线性表。

线性表	**栈**	**队列**
Insert(L, **i**, x)	Insert(S, **n+1**, x)	Insert(Q, **n+1**, x)
1≤i≤n+1		
Delete(L, **i**)	Delete(S, **n**)	Delete(Q, **1**)
1≤i≤n		

栈和队列是两种常用的数据类型

栈的定义

栈(Stack)是限制在表的一端进行插入和删除运算的线性表，通常称插入、删除的这一端为栈顶(Top)，另一端为栈底(Bottom)。当表中没有元素时称为空栈。

假设栈S=(a_1，a_2，a_3，...a_n)，则a_1称为栈底元素，a_n为栈顶元素。栈中元素按a_1，a_2，a_3，...a_n的次序进栈，退栈的第一个元素应为栈顶元素。因此，栈的修改是按后进先出的原则进行的，所以，栈称为后进先出（先进后出）表（LIFO，FILO）。

First In Last Out

抽象数据类型栈的定义

ADT Stack {

数据对象:

$D = \{ a_i \mid a_i \in ElemSet, i=1,2,...,n, n \geq 0 \}$

数据关系:

$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, i=2,...,n \}$

约定a_n 端为栈顶， a_1 端为栈底。

基本操作:

} ADT Stack

InitStack(&S)

DestroyStack(&S)

StackLength(S)

StackEmpty(S)

GetTop(S, &e)

ClearStack(&S)

Push(&S, e)

Pop(&S, &e)

StackTravers(S, visit())

InitStack(&S)

操作结果：构造一个空栈 S。

DestroyStack(&S)

初始条件：栈 S 已存在。

操作结果：栈 S 被销毁。

```
Status InitStack (SqStack &S, int maxsize)
{
    // 构造一个最大空间为 maxsize 的空顺序栈 S
  S.base = new ElemType[maxsize];
  if (!S.base) exit (OVERFLOW); //存储分配失败
  S.top = S.base;
  S.stacksize = maxsize;
  return OK;
}
```

StackEmpty(S)

初始条件：栈 S 已存在。

操作结果：若栈 S 为空栈，则返回 TRUE，否则 FALSE。

StackLength(S)

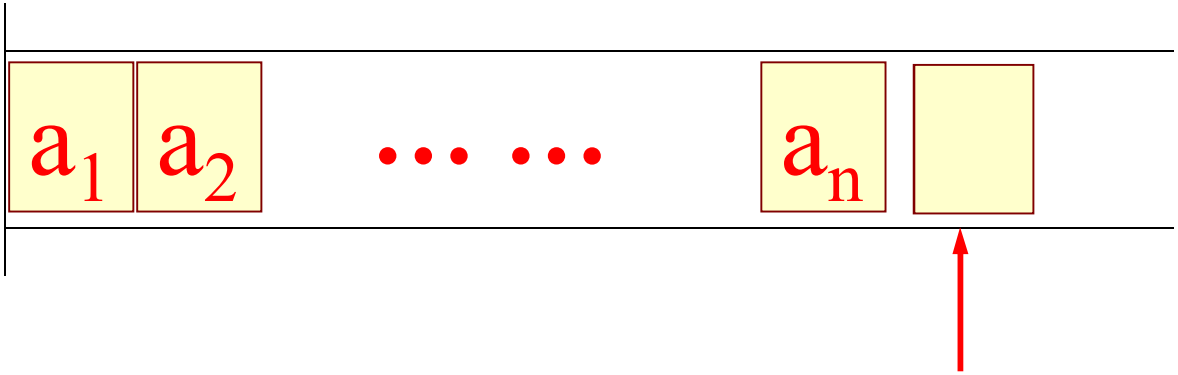
初始条件：栈 S 已存在。

操作结果：返回 S 的元素个数，即栈的长度。

GetTop(S, &e)

初始条件：栈 S 已存在且非空。

操作结果：用 e 返回 S 的栈顶元素。



ClearStack(&S)

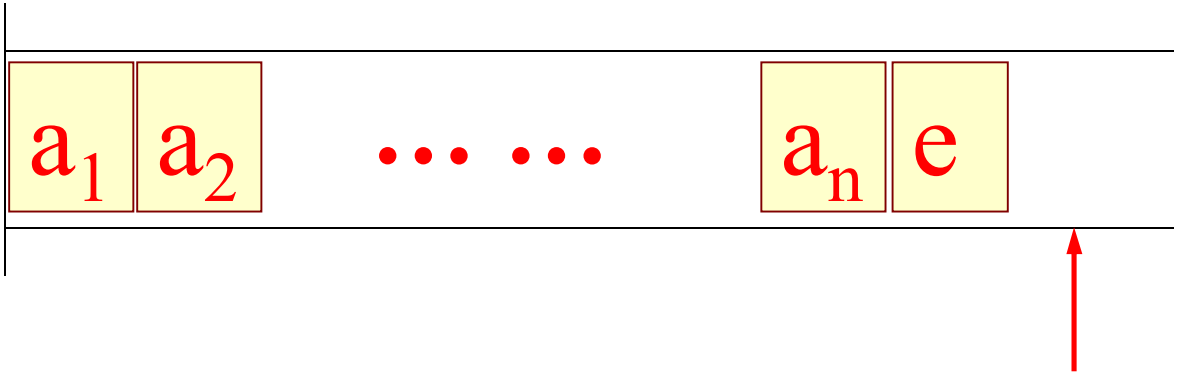
初始条件：栈 S 已存在。

操作结果：将 S 清为空栈。

Push(&S, e)

初始条件：栈 S 已存在。

操作结果：插入元素 e 为新的栈顶元素。

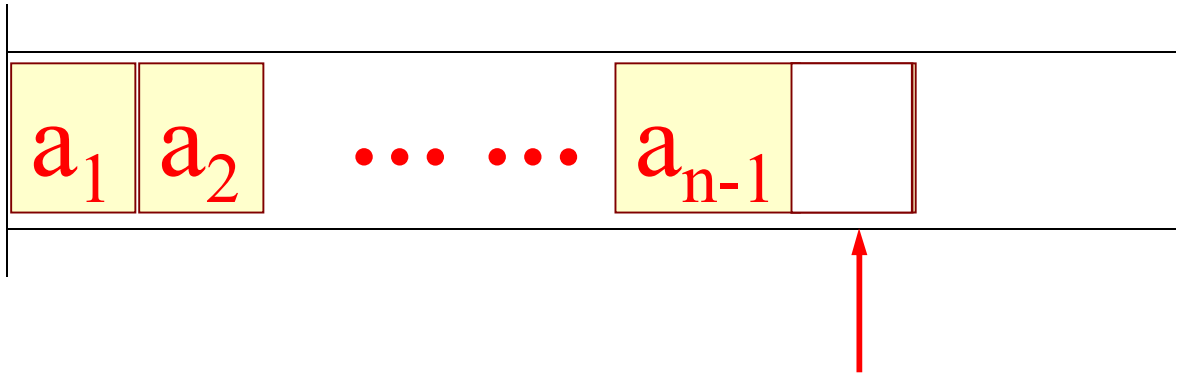


```
Status Push (SqStack &S, SElemType e)
{ // 若栈不满，则将 e 插入栈顶
  if (S.top - S.base >= S.stacksize)  //栈满
    return OVERFLOW;
  *S.top++ = e;
  return OK;
}
```

Pop(&S, &e)

初始条件：栈 S 已存在且非空。

操作结果：删除 S 的栈顶元素，并用 e 返回其值。



```
Status Pop (SqStack &S, SElemType &e) {
    // 若栈不空，则删除S的栈顶元素，
    // 用 e 返回其值，并返回OK；
    // 否则返回ERROR
    if (S.top == S.base) return ERROR;
    e = *--S.top;
    return OK;
}
```

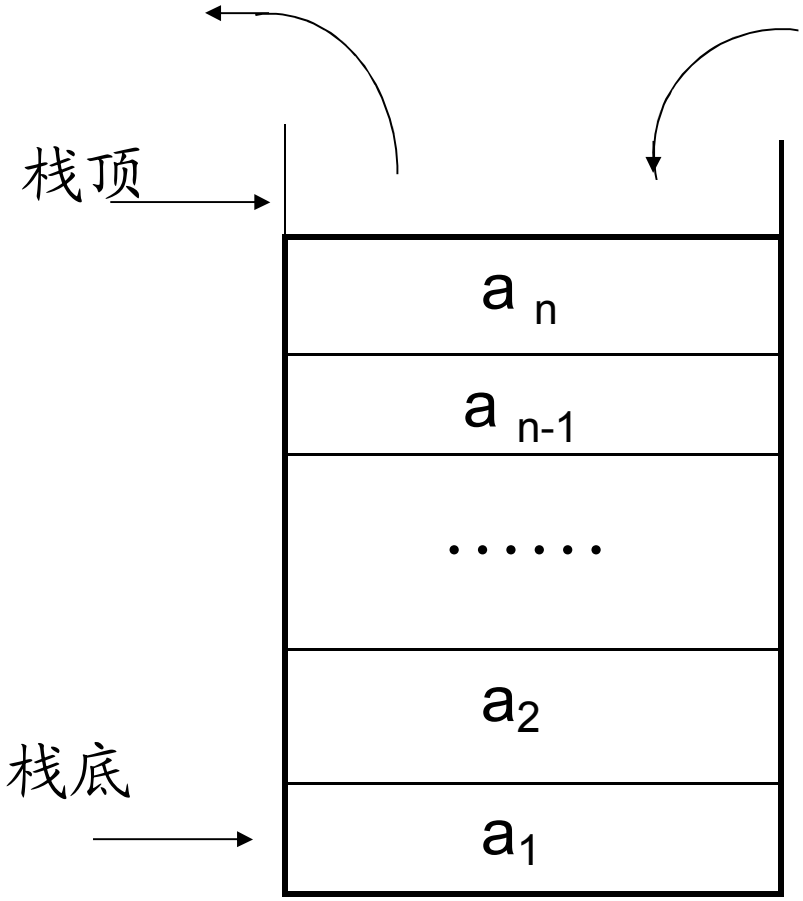
3.1.2 栈的表示和实现

栈是线性表的特例，因此线性表的存储结构对栈也适应。

栈的顺序存储结构简称为顺序栈，可用数组来实现顺序栈。因为栈底位置是固定不变的，所以可以将栈底位置设置在数组的两端的任何一个端点；栈顶位置是随着进栈和退栈操作而变化的，故需用一个整型变量top（栈顶指针）来指出栈顶。

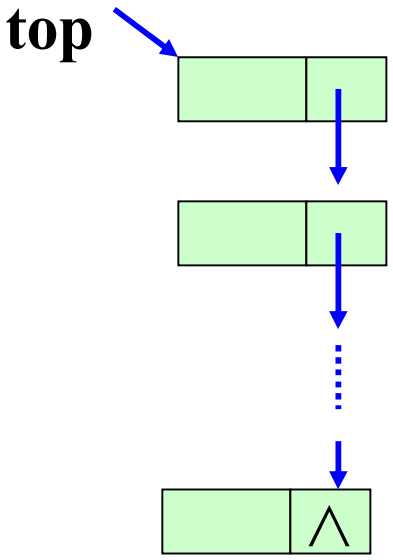
顺序存储栈例子

例：一叠盘子、弹夹等。



链栈

栈的链式存储结构称为链栈，插入和删除操作仅限制在链头位置上进行。栈顶指针就是链表的头指针。



初始化

```
top=NULL
```

x 进栈

```
p= new StackNode;
p->data=x;
p->next=top;
top=p;
```

退栈

```
y= top->data;
p=top;
top=p->next;
delete p;
```

例 指出下述程序段的功能是什么？

```
(1) void Demo1(SeqStack *S){
        int i; arr[64] ; n=0 ;
        while (!StackEmpty(S))
            arr[n++]=Pop(S);
        for (i=0, i< n; i++)
            Push(S, arr[i]);
    } //Demo1
```

```
(2) SeqStack S1, S2, tmp;
    DataType x;
    ... //假设栈tmp和S2已做过初始化
    while ( ! StackEmpty (&S1)) {
        x=Pop(&S1) ;
        Push(&tmp,x);
    }
    while ( ! StackEmpty (&tmp) ){
        x=Pop( &tmp);
        Push( &S1,x);
        Push( &S2, x);
    }
```

```
(3) void Demo2( SeqStack *S, int m) {
        // 设DataType 为int 型
        SeqStack T; int i;
        InitStack (&T);
        while (! StackEmpty( S))
            if(( i=Pop(S)) !=m)
                Push( &T,i);
        while (! StackEmpty( &T)){
            i=Pop(&T);
            Push(S,i);
        }
    }
```

(1) 程序段的功能是将一栈中的元素按反序重新排列，也就是原来在栈顶的元素放到栈底，栈底的元素放到栈顶。此栈中元素个数限制在**64**个以内。

(2)程序段的功能是利用**tmp**栈将一个非空栈的所有元素按原样复制到一个空栈当中去。

(3)程序段的功能是将一个非空栈中值等于**m**的元素全部删去。

第三章 栈和队列

3.2 栈的应用举例

- 数制转换
- 括号匹配的检验
- 行编辑程序
- 迷宫求解
- 表达式求值（自己课下看）

3.2 栈的应用举例

由于栈结构具有的后进先出的固有特性，致使栈成为程序设计中常用的工具。以下是几个栈应用的例子。

数制转换
括号匹配的检验
行编辑程序
迷宫求解
表达式求值

3.2.1 数制转换

十进制N和其它进制数的转换是计算机实现计算的基本问题，其解决方法很多，其中一个简单算法基于下列原理：

N=(n / d)*d+n % d

例如：$(1348)_{10}=(2504)_{8}$，其运算过程如下：

n	n /8	n % 8
1348	168	4
168	21	0
21	2	5
2	0	2

```
void conversion( )
{
        InitStack(s);
        cin>>n;
        while(n)
        {
            push(s,n%8);
            n=n/8;
        }
        while(! StackEmpty(s))
        {
            pop(s,e);
              cout<<e;
        }
}
```

数制转换C++代码

```
#include <iostream.h>
#define STACK_INIT_SIZE 100
#define TRUE 1
#define FALSE 0
struct SqStack{
    int *base;
    int *top;
    int stacksize;
};
```

```
void DestroyStack(SqStack &s){
    delete s.base;
}
void push(SqStack &s, int e){
    *s.top++=e;
}
void pop(SqStack &s, int &e){
    e=*--s.top;
}
```

```
void InitStack(SqStack &s){
    s.stacksize=STACK_INIT_SIZE;
    s.base=new int[s.stacksize];
    s.top=s.base;
}
bool StackEmpty(SqStack &s){
    if(s.top==s.base)
        return TRUE;
    else
        return FALSE;
}
```

DecimalOctal.cpp

```
void main(){
    int n, e;
    SqStack s;
    InitStack(s);
    cin>>n;
    while(n){
        push(s,n%8); n=n/8;
    }
    while(! StackEmpty(s)){
        pop(s, e); cout<<e;
    }
    DestroyStack(s);
}
```

DecimalOctal.cpp

3.2.2 括号匹配的检验

假设表达式中允许括号嵌套,则检验括号是否匹配的方法可用栈。例：

（（）（）（（））

假设在表达式中

（ [ ] （ ） ）或 [（ [ ] [ ] ）]

等为正确的格式，

[（] ）或（ [（ ） ）或 (( )])

均为不正确的格式。

则 检验括号是否匹配的方法可用

“期待的急迫程度”这个概念来描述。

例如：考虑下列括号序列：

[([ ] [ ])]

1 2 3 4 5 6 7 8

分析可能出现的不匹配的情况：

- 到来的右括弧不是所“期待”的；
- 到来的是“不速之客”；
- 直到结束，也没有到来所“期待”的括弧；

算法的设计思想：

1）凡出现左括弧，则进栈；

2）凡出现右括弧，首先检查栈是否空

若栈空，则表明该“右括弧”多余

否则和栈顶元素比较，

若相匹配，则“左括弧出栈”

否则表明不匹配

3）表达式检验结束时，

若栈空，则表明表达式中匹配正确

否则表明“左括弧”有余

```
Status matching(string& exp) {
  int state = 1; int i = 1;
  while (i<=Length(exp) && state) {
    switch of exp[i] {
      case 左括弧:{Push(S,exp[i]); i++; break;}
      case ”)”: {
        if(NOT StackEmpty(S)&&GetTop(S)==“(“
          {Pop(S,e);  i++;}
        else {state = 0;}
        break;  }   … …
}
if (StackEmpty(S)&&state) return OK; ……
```

括号匹配检验C++代码

```
#include <iostream.h>
#define STACK_INIT_SIZE 100
#define TRUE 1
#define FALSE 0
struct SqStack{
    int *base;
    int *top;
    int stacksize;
};
```

```
void push(SqStack &s, int e){
    *s.top++=e;
}
void pop(SqStack &s, int &e){
    e=*--s.top;
}
```

```
void InitStack(SqStack &s){
    s.stacksize=STACK_INIT_SIZE;
    s.base=new int[s.stacksize];
    s.top=s.base;
}
bool StackEmpty(SqStack &s){
    if(s.top==s.base)  return TRUE;
    else  return FALSE;
}
 void DestroyStack(SqStack &s){
    delete s.base;
}
```

PairMatch.cpp

```
void main(){
      char expr[STACK_INIT_SIZE];
      cin.getline(expr, STACK_INIT_SIZE);
      int i, j, length=strlen(expr);
      SqStack s;  InitStack(s);
      for(i=0; i<length; i++) {
          if(expr[i]=='(') push(s, i+1);
          else if(expr[i]==')') {
             if(!StackEmpty(s)) {
                  pop(s, j);
                     cout<<"("<<j<<", "<<i+1<<")"<<endl;}
             else
                  cout<<"No match at  "<<i+1<<endl;
          }
       }
       while(!StackEmpty(s)) {
           pop(s, j);  cout<<"No match at  "<<j<<endl;
       }
       DestroyStack(s);
}
```

例三、行编辑程序问题

如何实现?

“每接受一个字符即存入存储器” ?

并不恰当!

在用户输入一行的过程中，允许用户输入出差错，并在发现有误时可以及时更正。

合理的作法是：

设立一个输入缓冲区，用以接受用户输入的一行字符，然后逐行存入用户数据区；并假设“#”为退格符，“@”为退行符。

假设从终端接受了这样两行字符:

```
whli##ilr#e（ s#*s)
    outcha@putchar(*s=#++);
```

则实际有效的是下列两行:

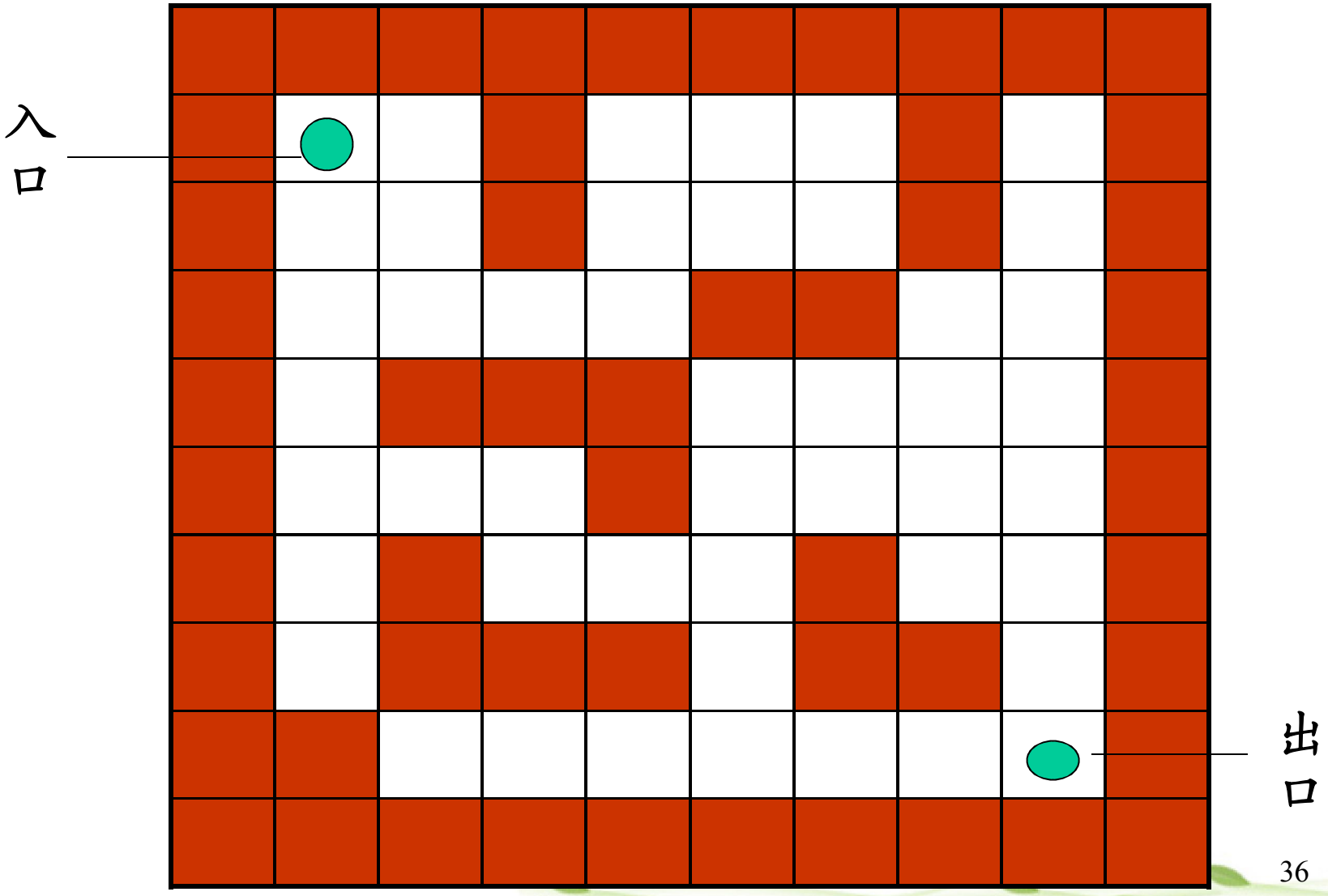
```
while (*s)
  putchar(*s++);
```

```
while (ch != EOF) { //EOF为全文结束符
  while (ch != EOF && ch != '\n') {
    switch (ch) {
     case '#' : Pop(S, c); break;
     case '@': ClearStack(S); break;// 重置S为空栈
     default : Push(S, ch);  break;
    }
    ch = getchar();  // 从终端接收下一个字符
  }
```

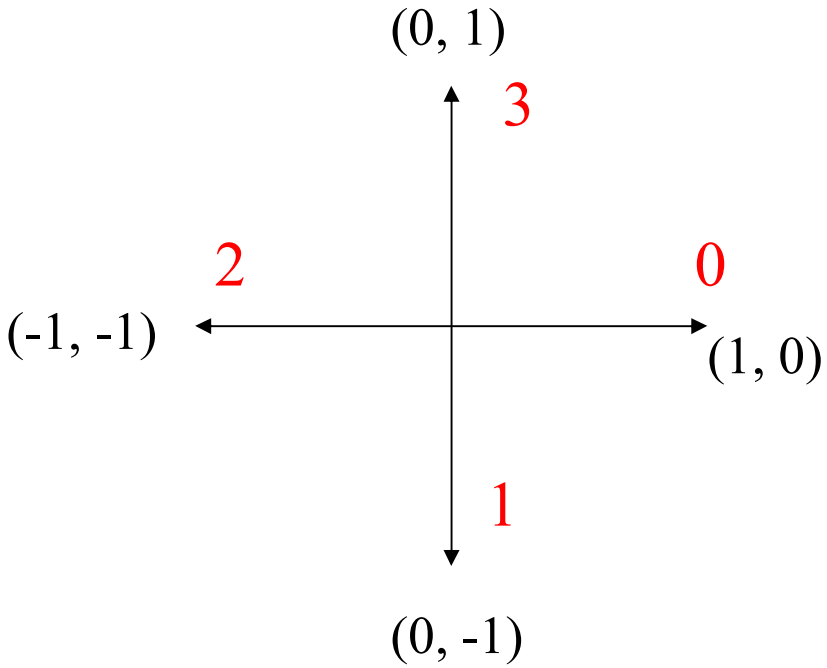
将从栈底到栈顶的字符传送至调用过程的数据区;

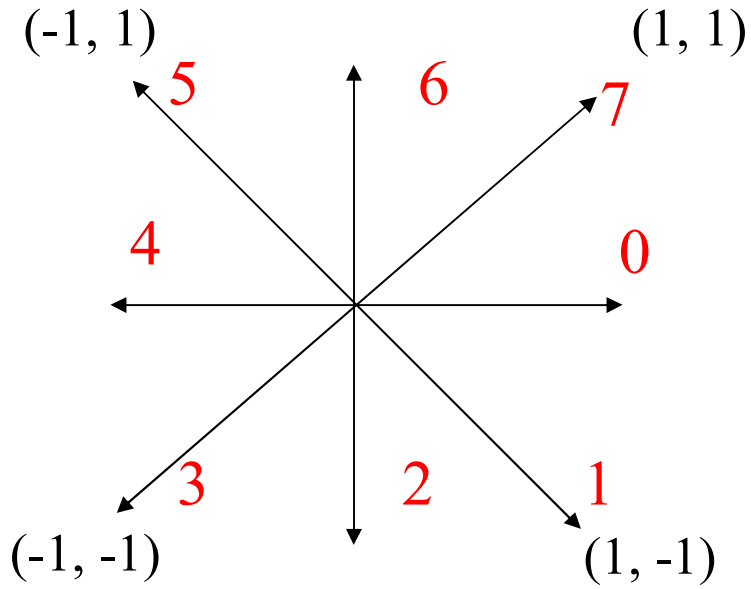
```
  ClearStack(S);      // 重置S为空栈
  if (ch != EOF)  ch = getchar();
```

3.2.4 迷宫求解



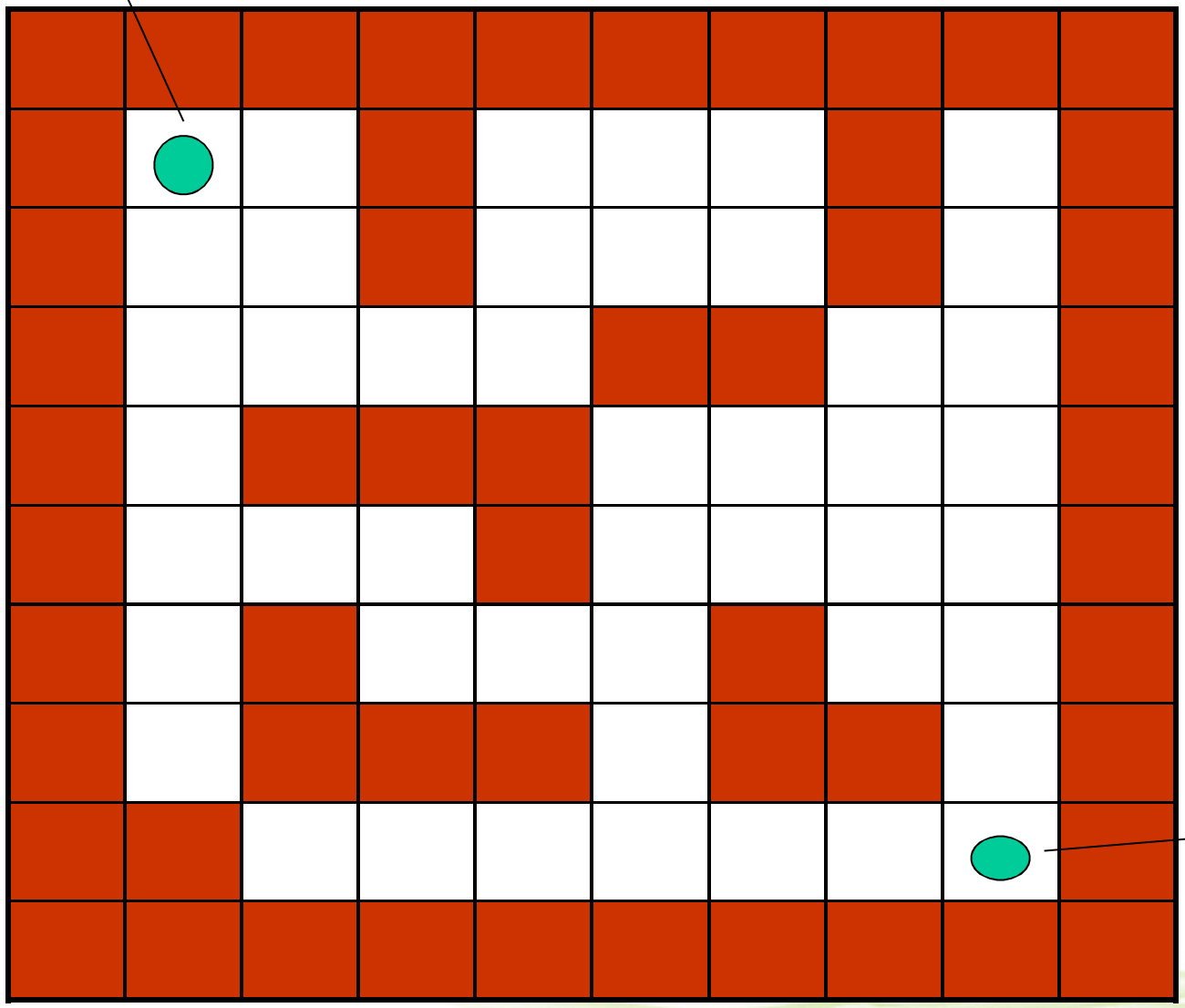
移动方向





4方向与8方向

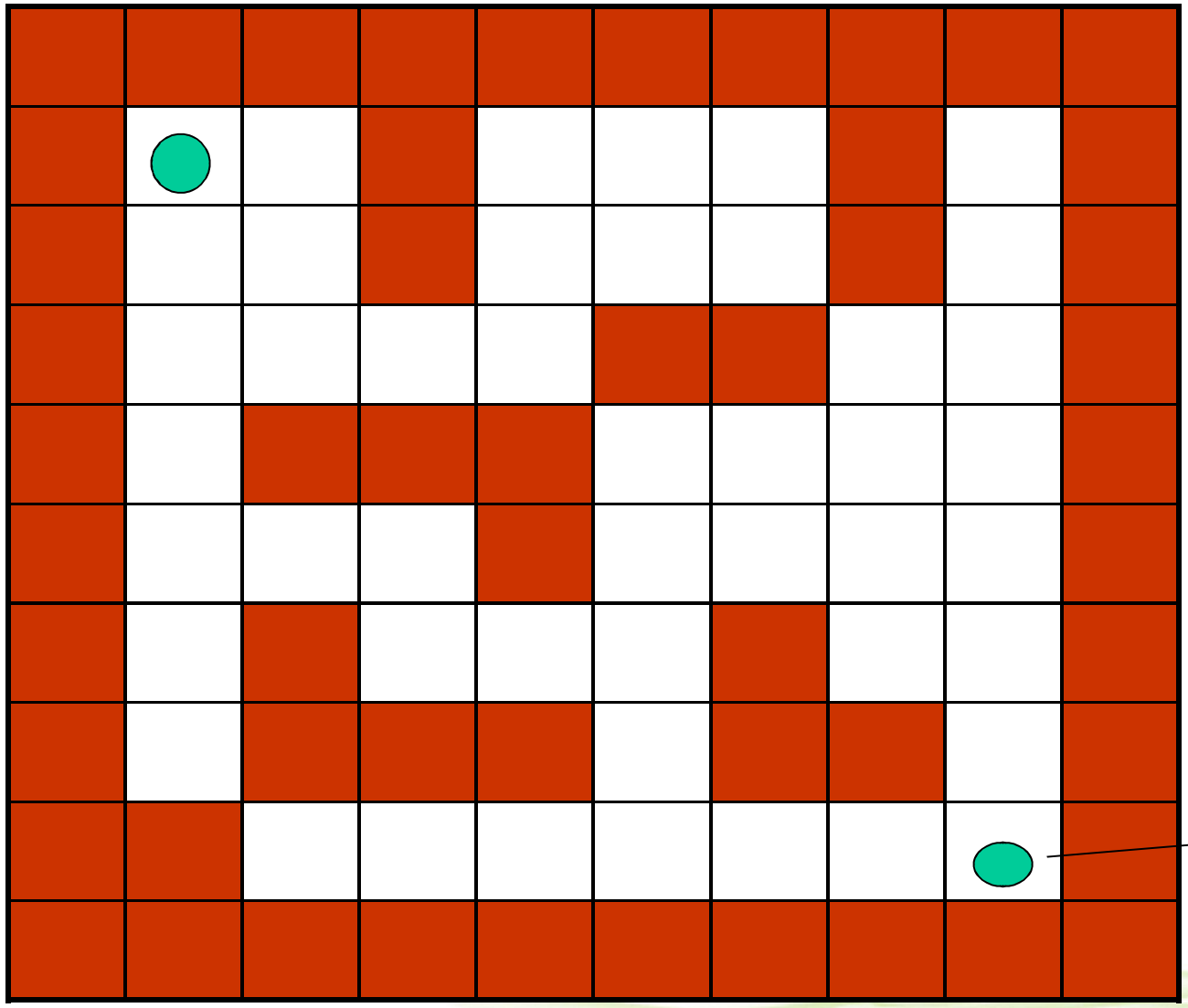
通常用的是“穷举求解”的方法



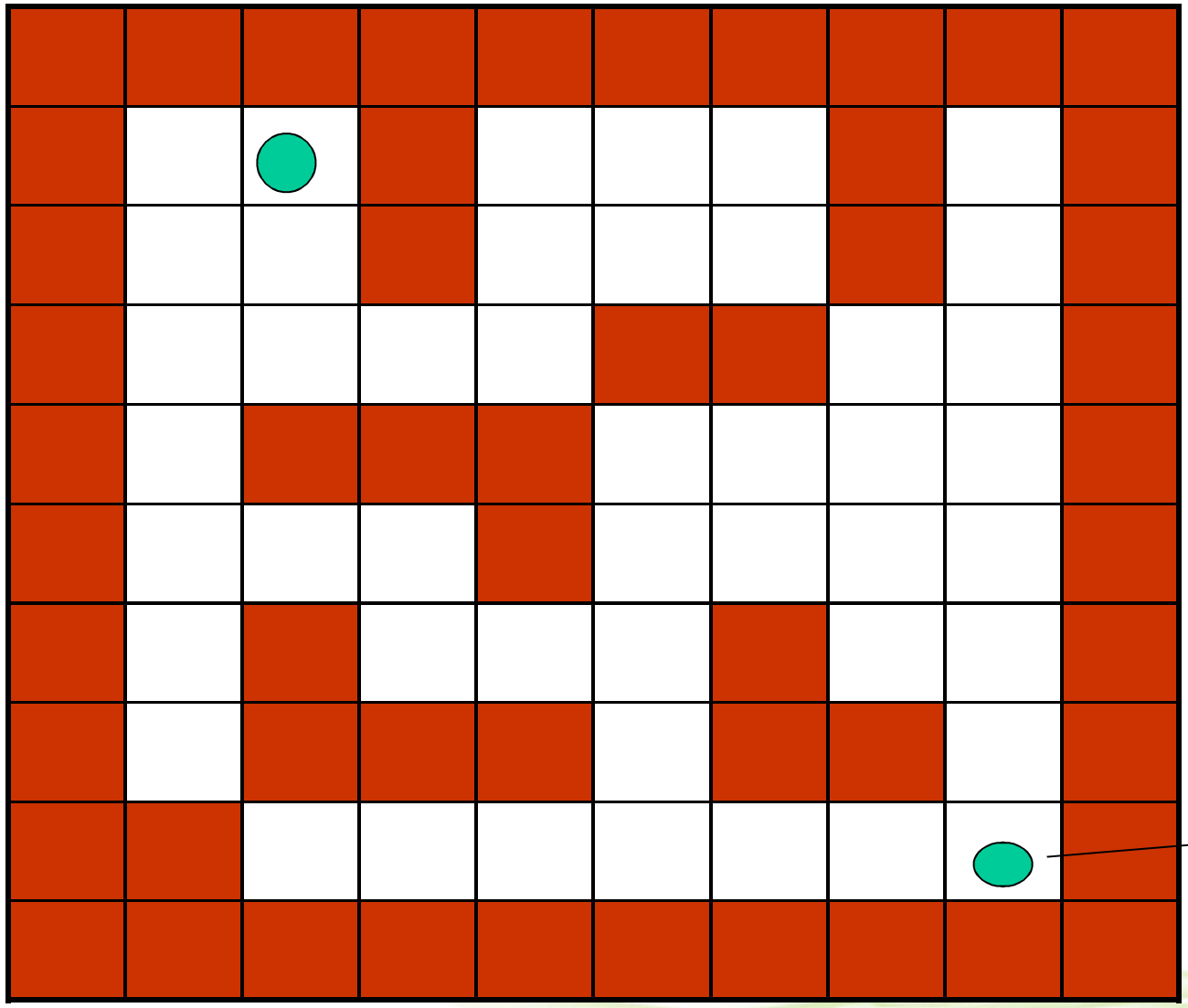
求迷宫路径算法的基本思想是:

- 若当前位置“可通”(未曾走到过的通道块)，则纳入路径，继续前进;
- 若当前位置“不可通”，则后退，换方向继续探索;
- 若四周“均无通路”，则将当前位置从路径中删除出去。

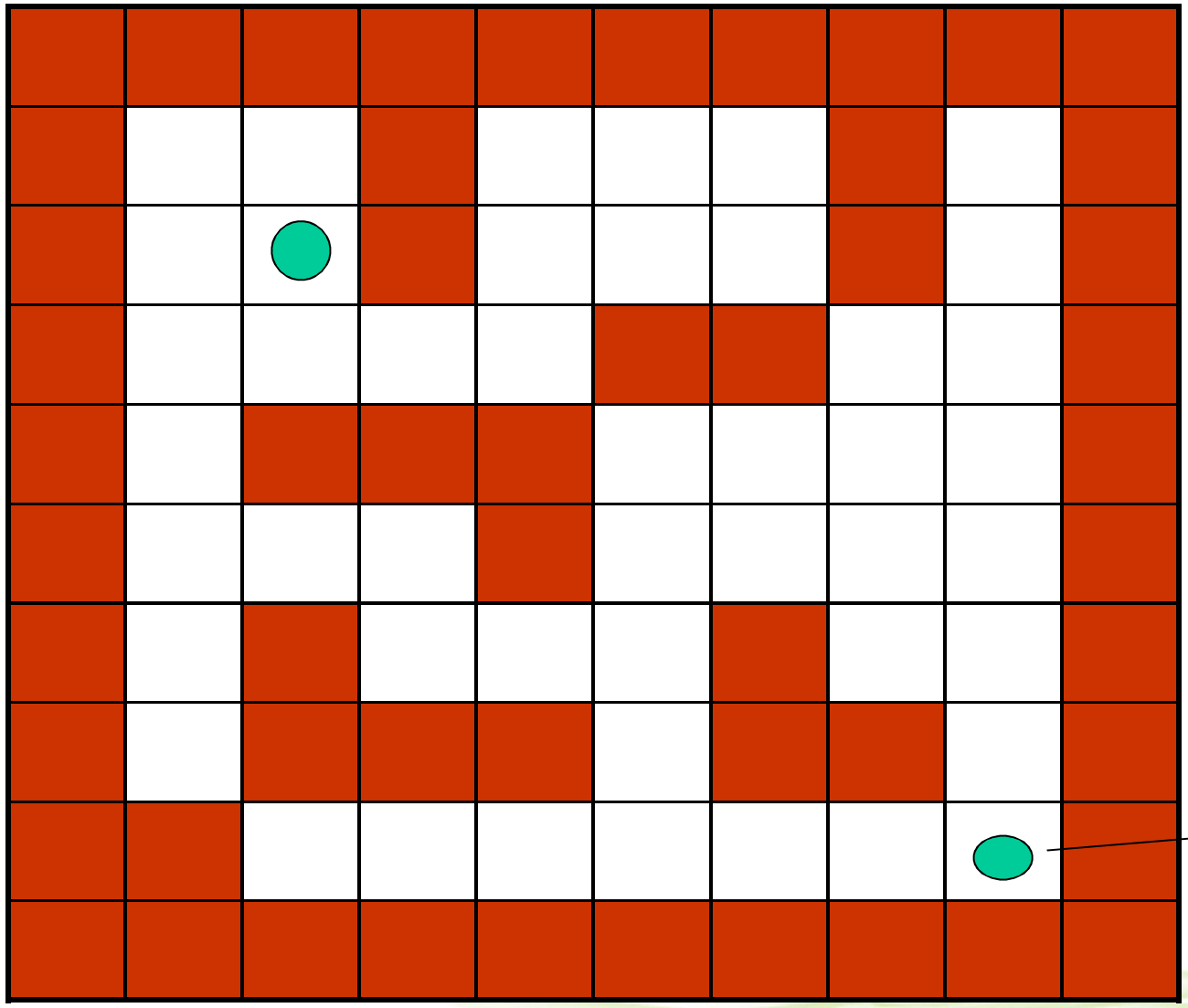
3.2.4 迷宫求解



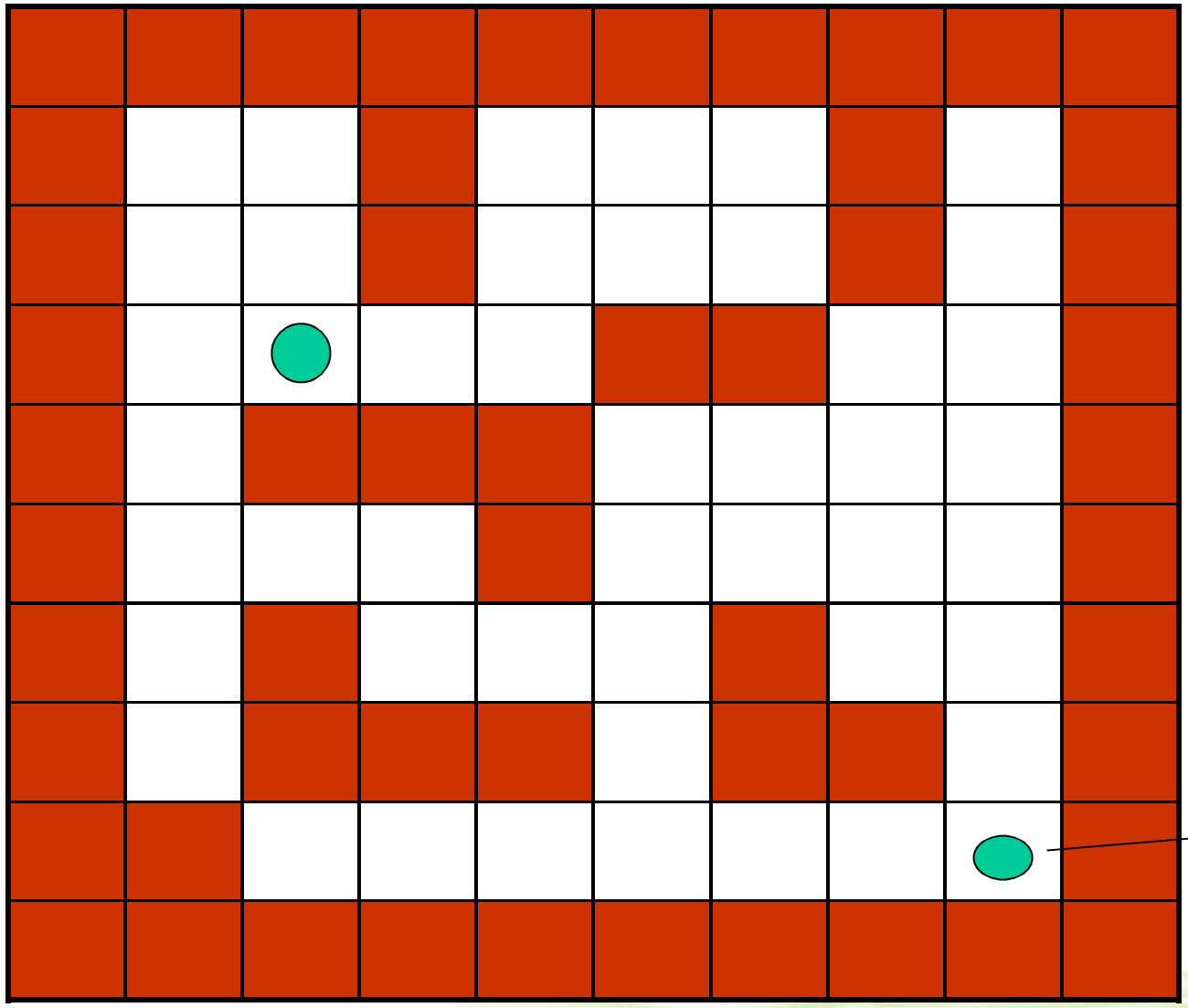
3.2.4 迷宫求解



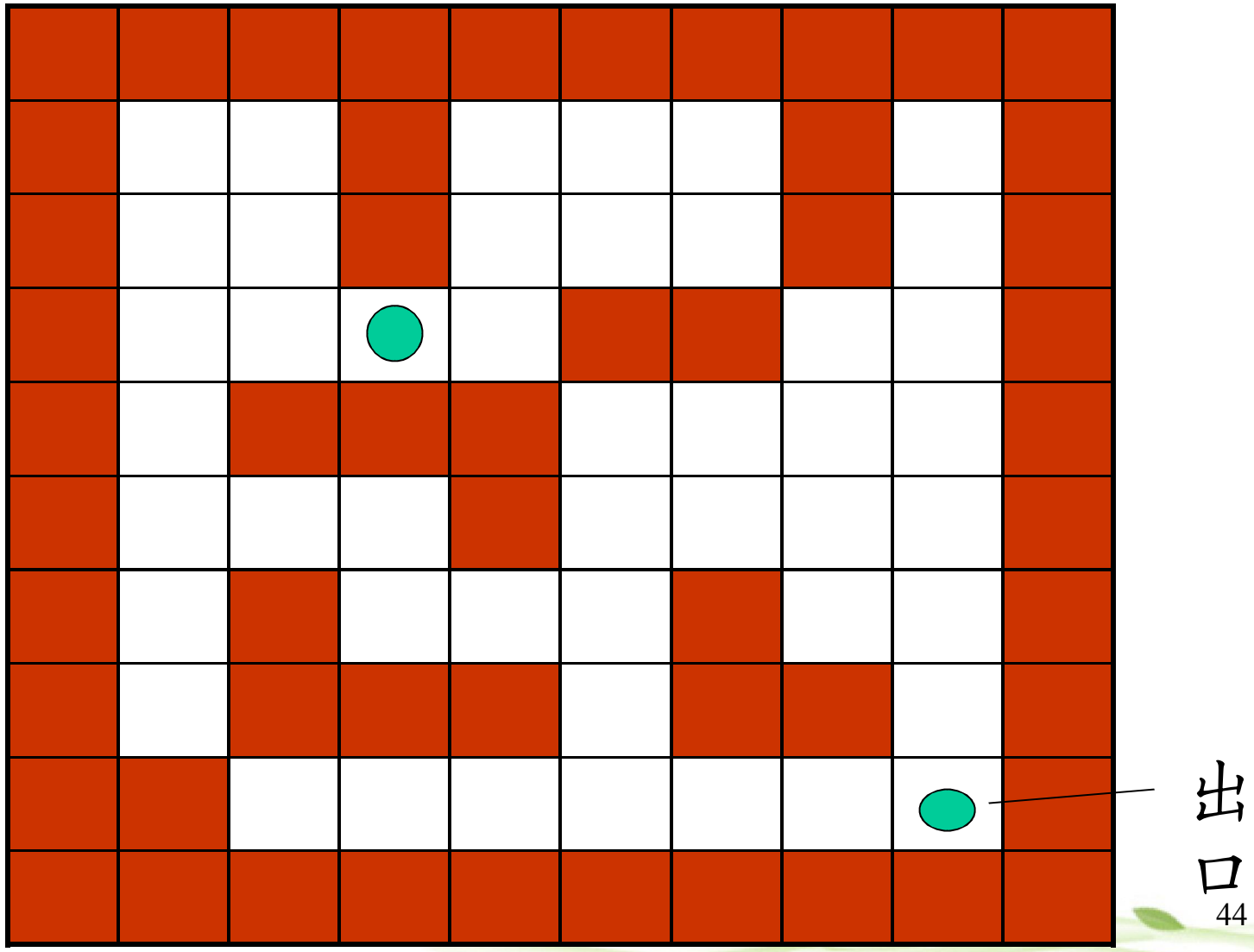
3.2.4 迷宫求解



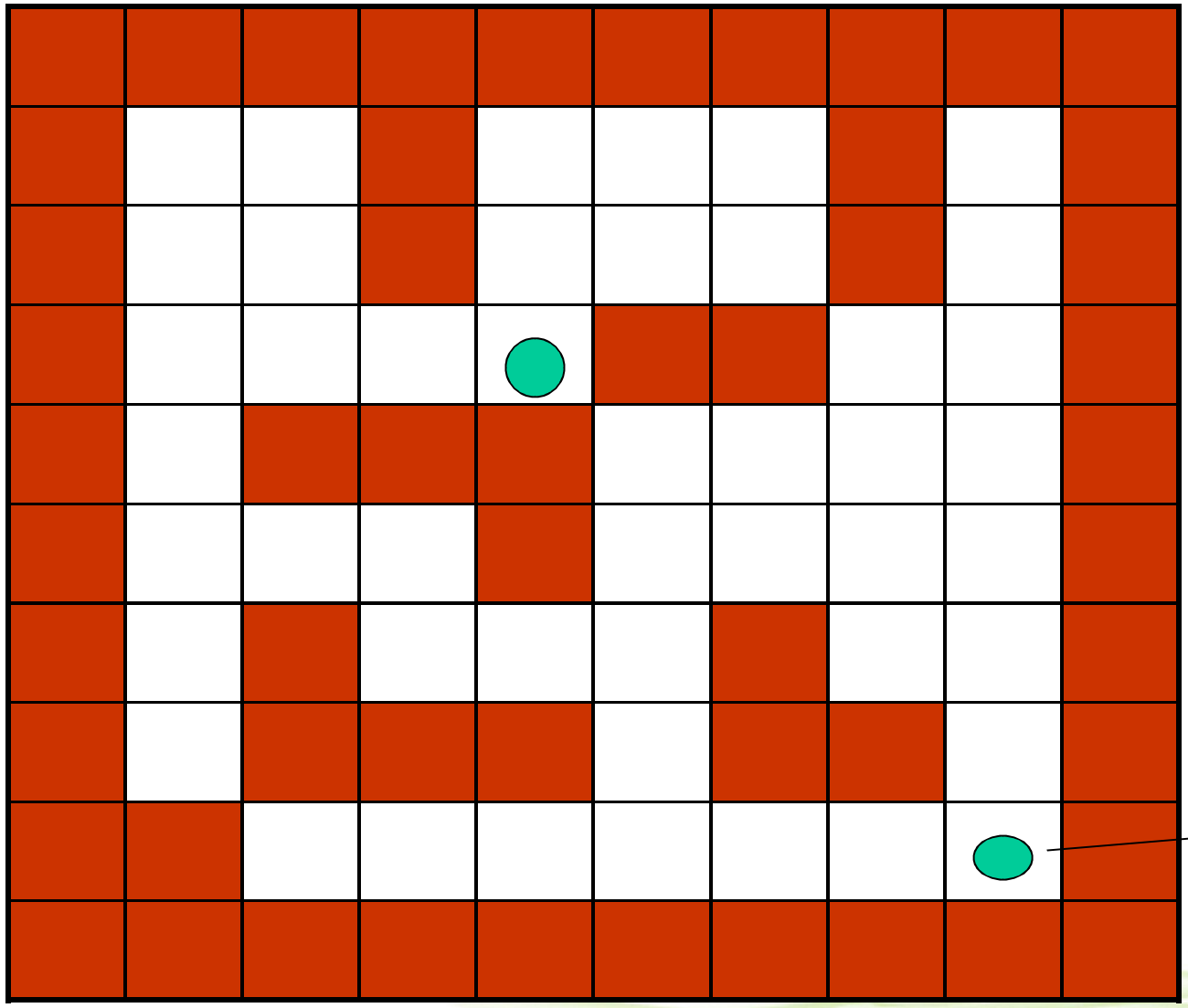
3.2.4 迷宫求解



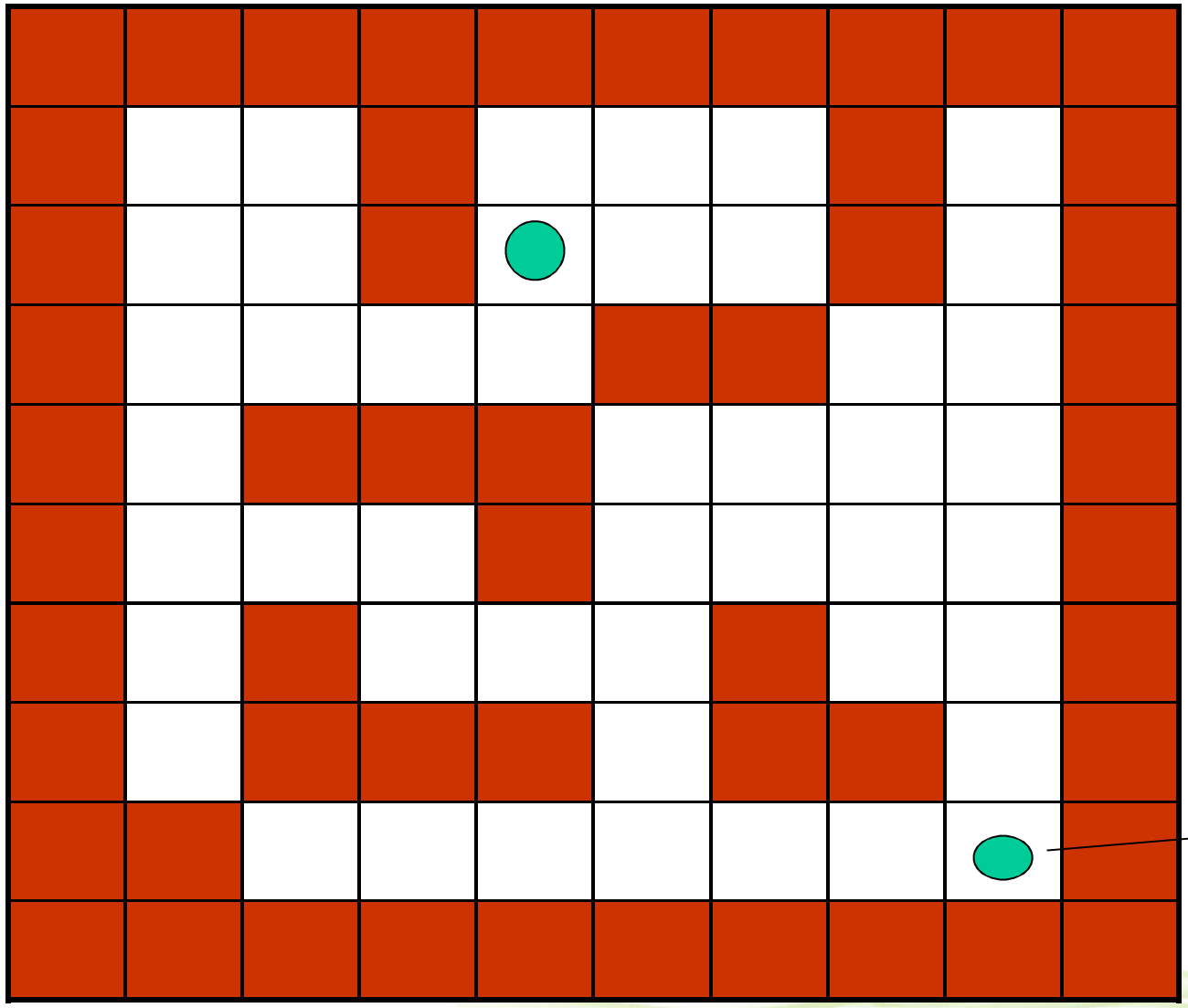
3.2.4 迷宫求解



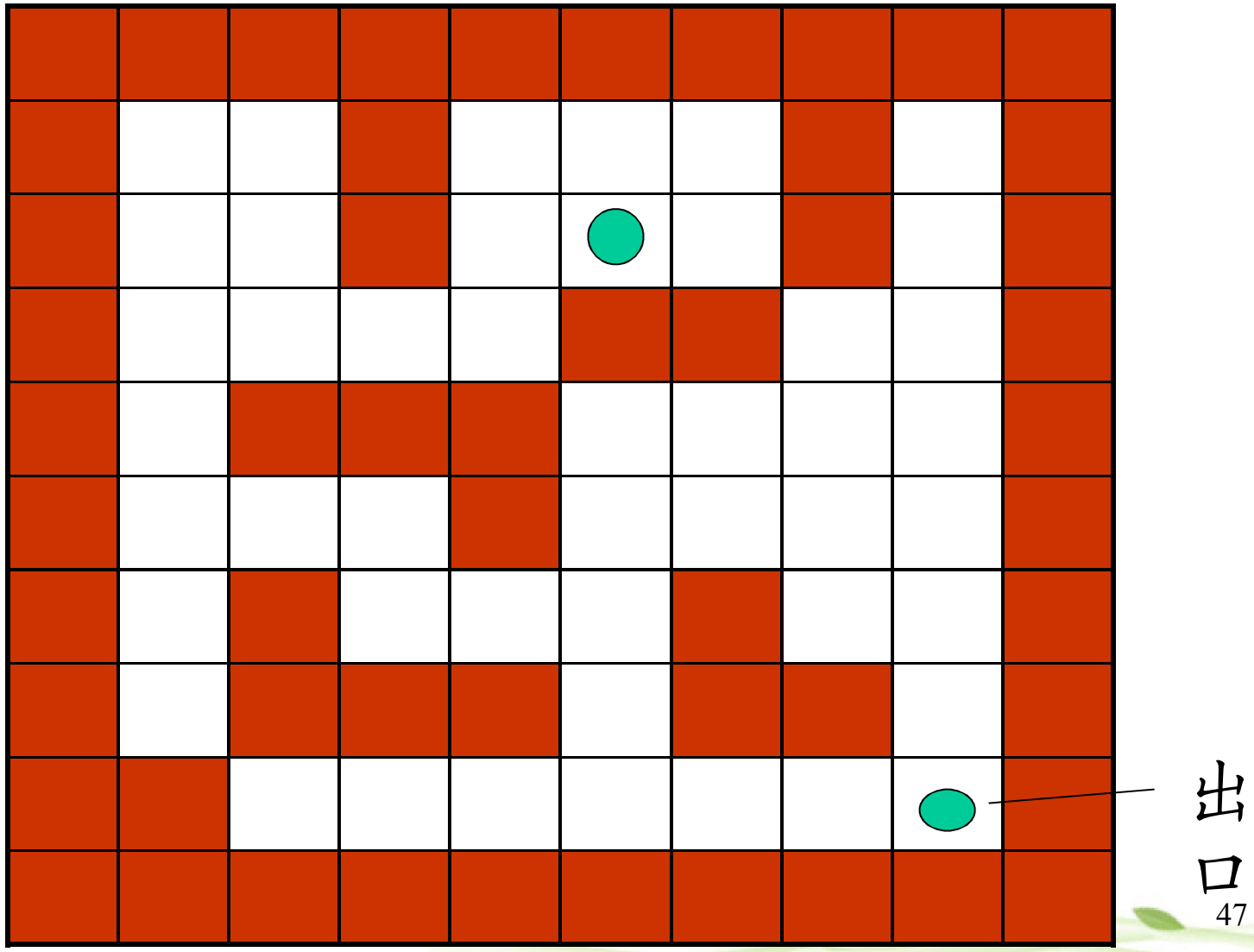
3.2.4 迷宫求解



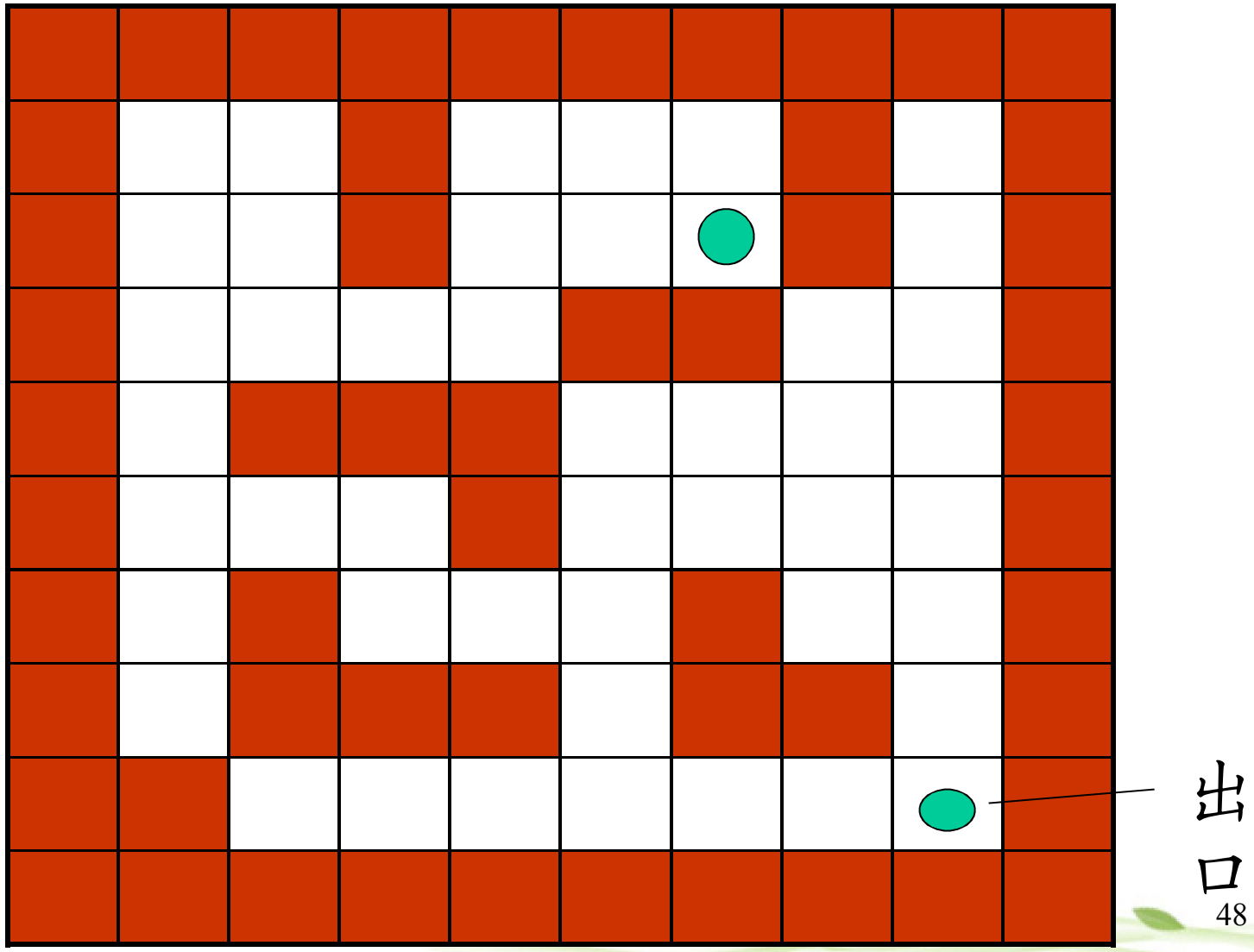
3.2.4 迷宫求解



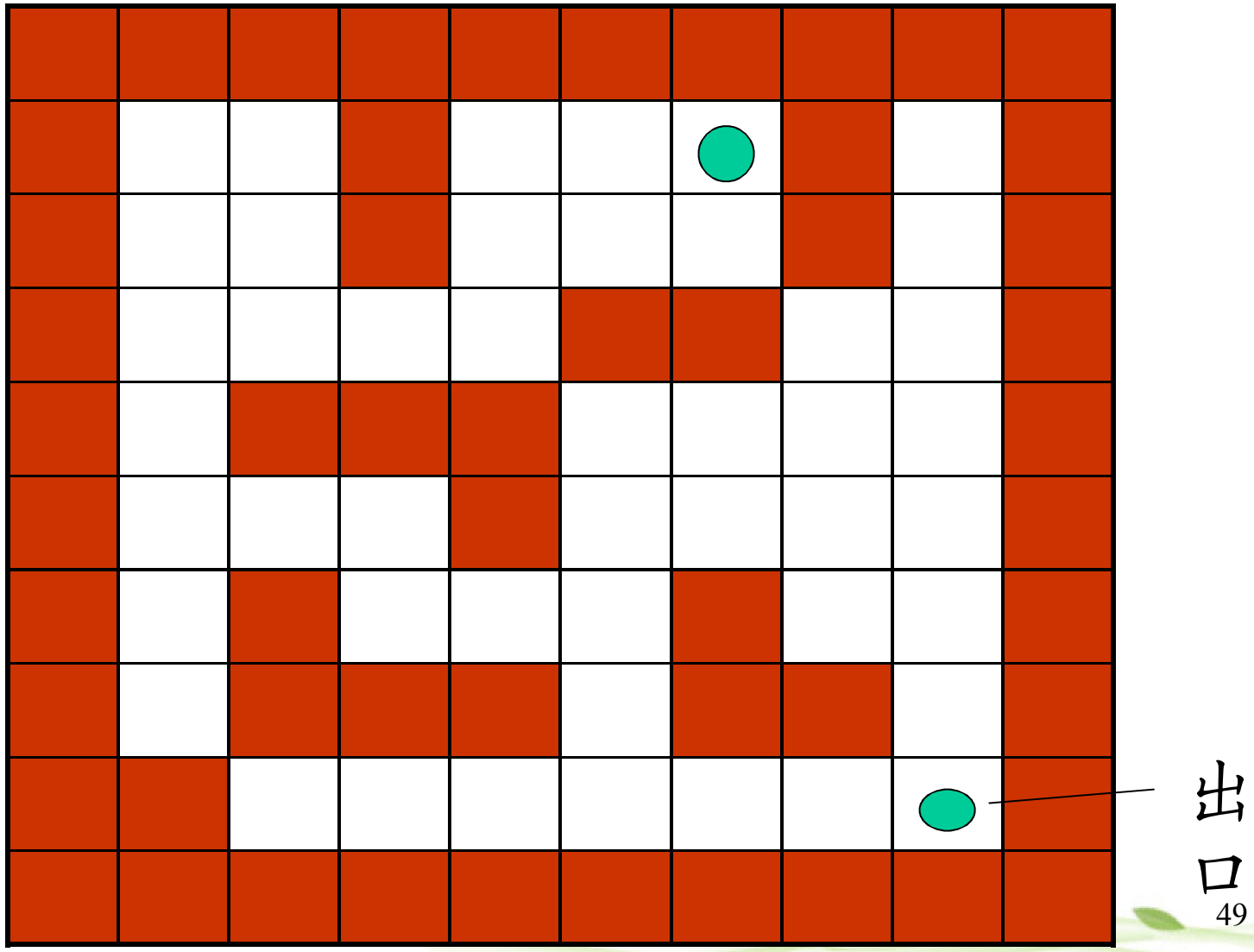
3.2.4 迷宫求解



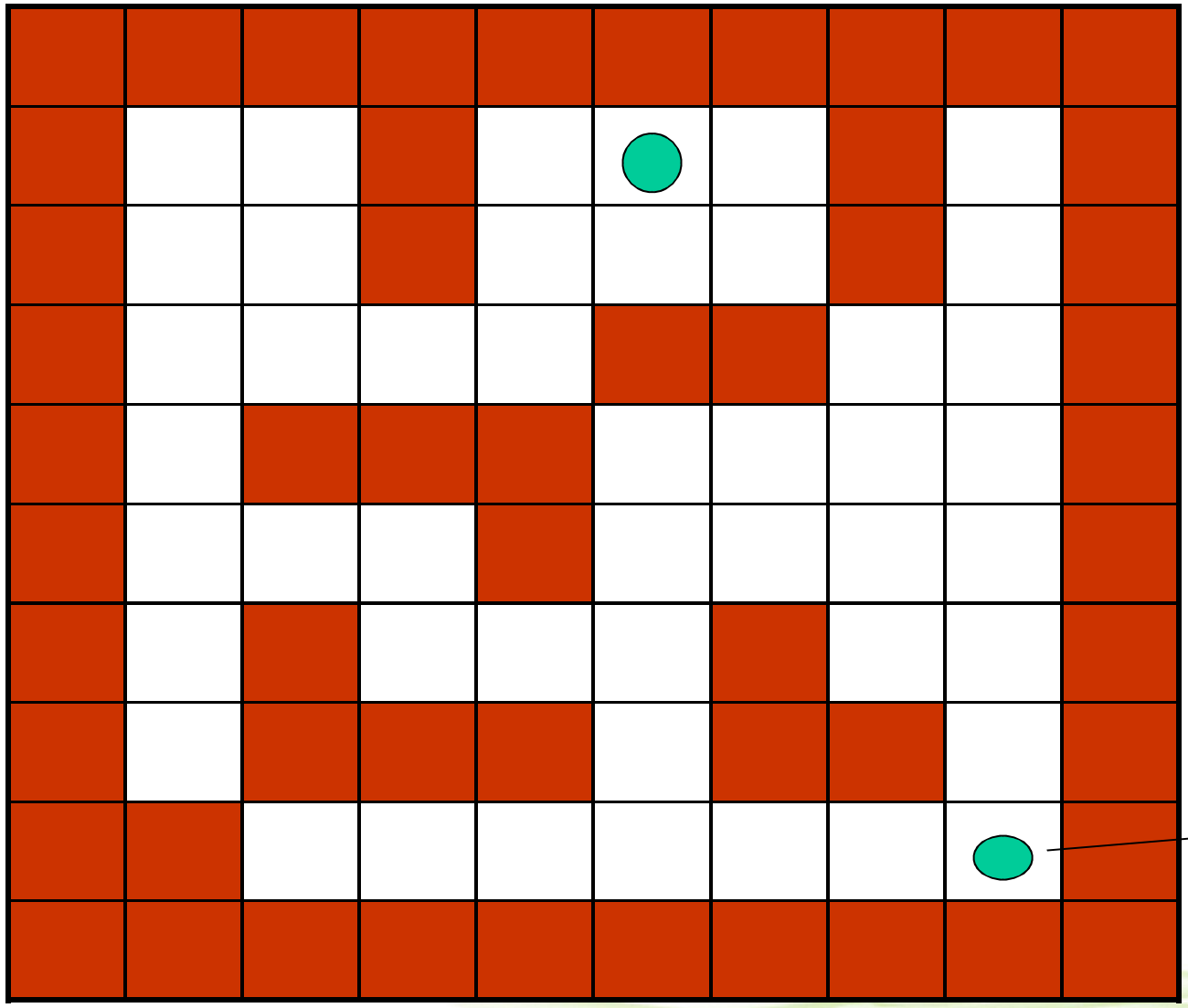
3.2.4 迷宫求解



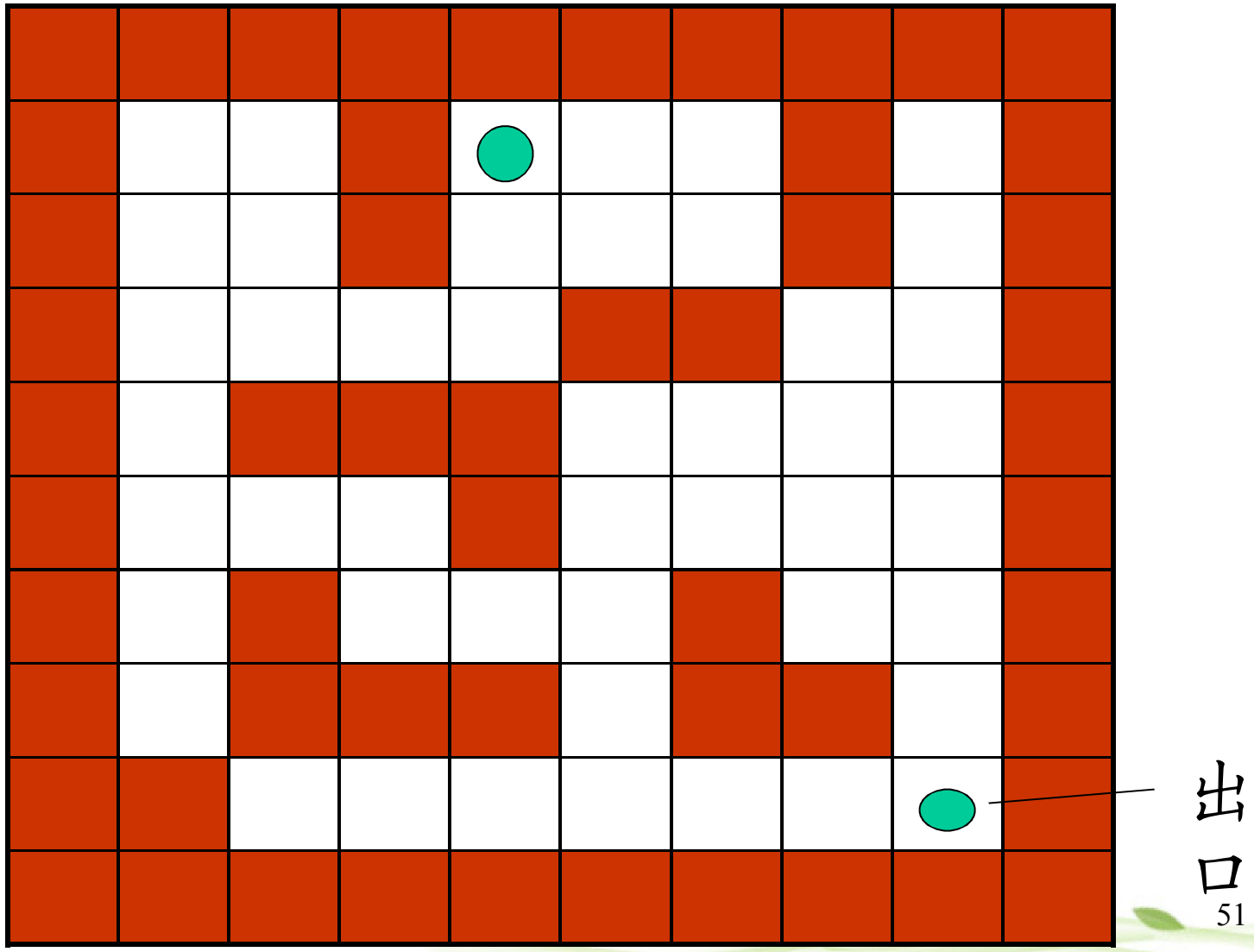
3.2.4 迷宫求解



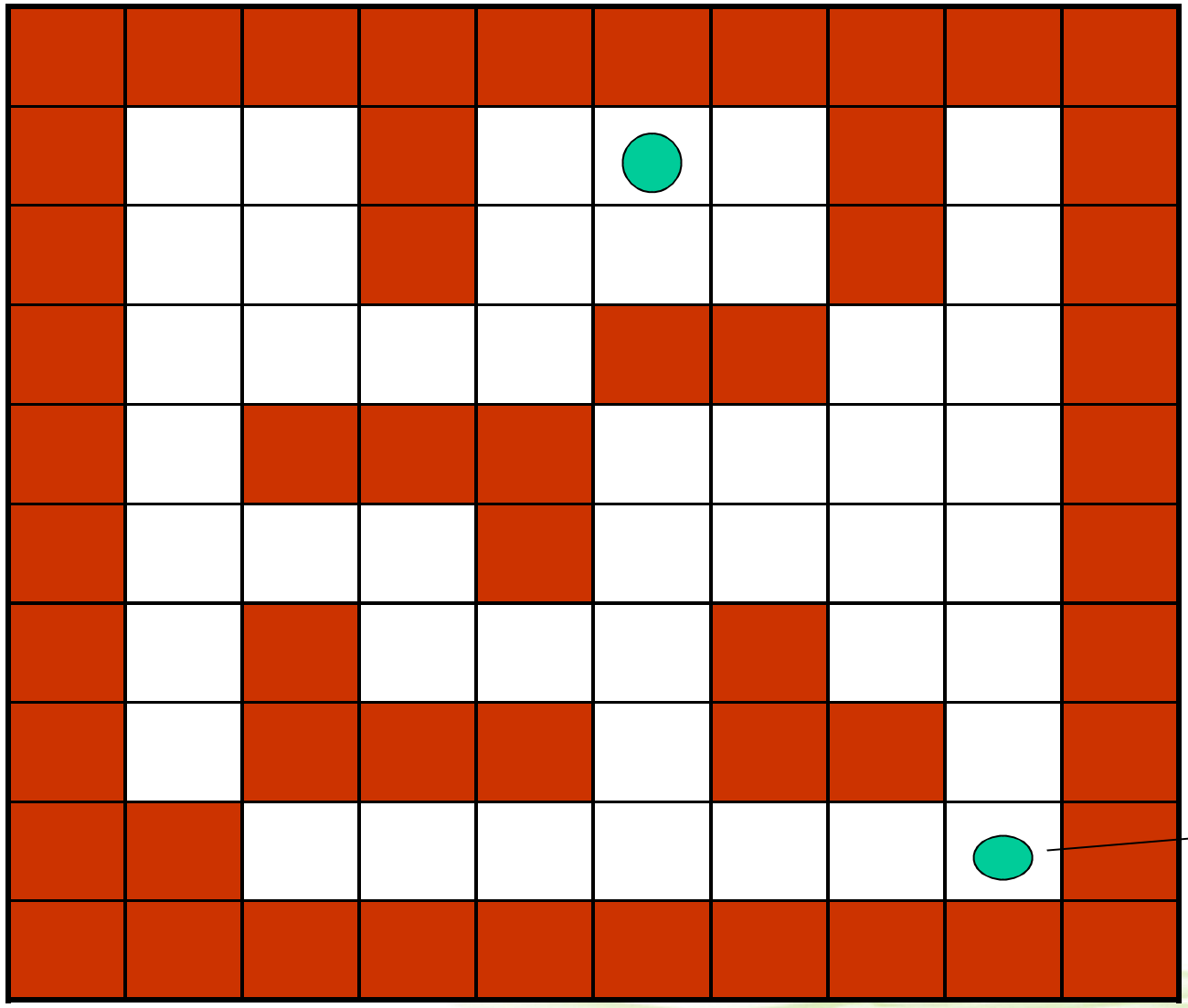
3.2.4 迷宫求解



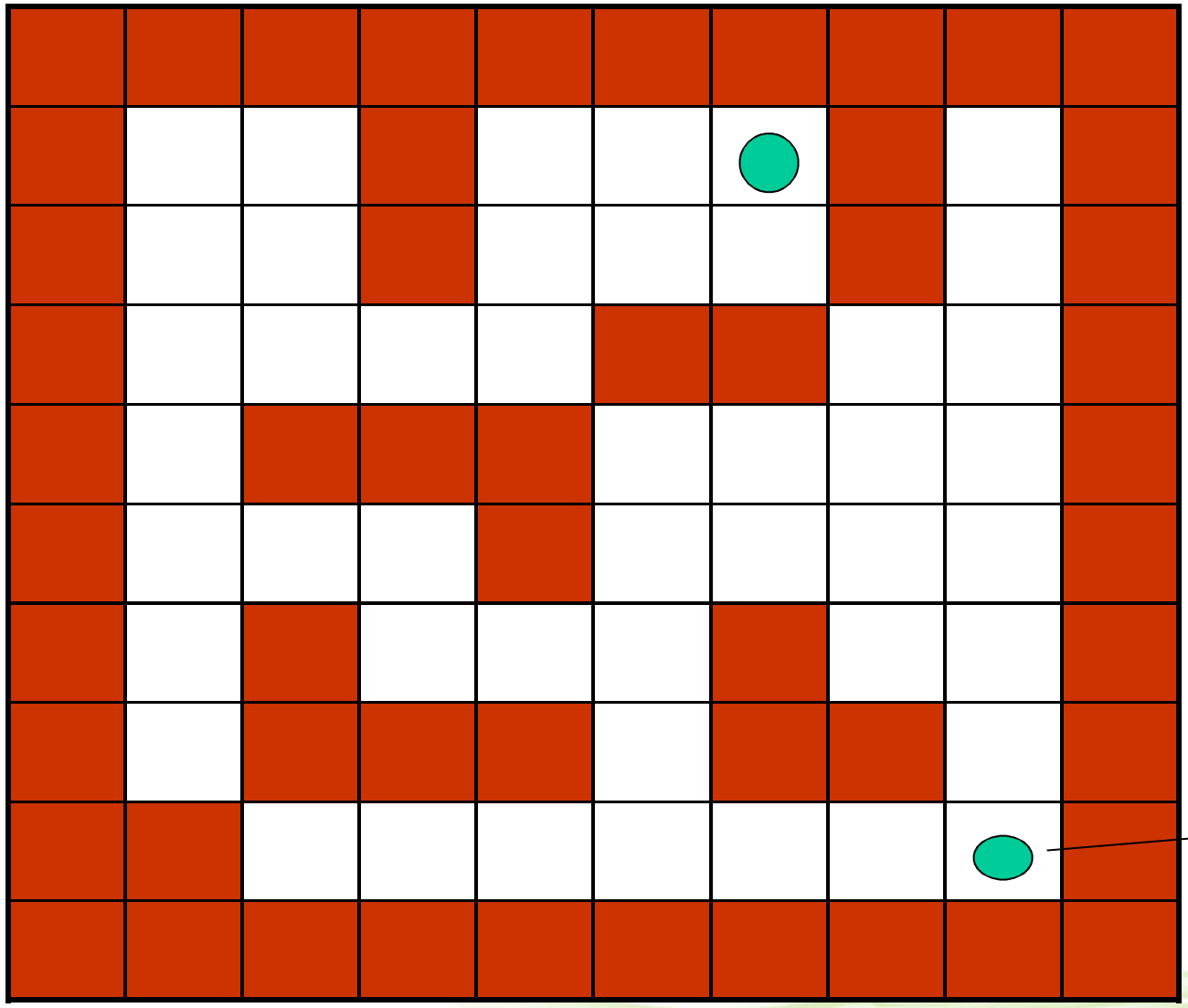
3.2.4 迷宫求解



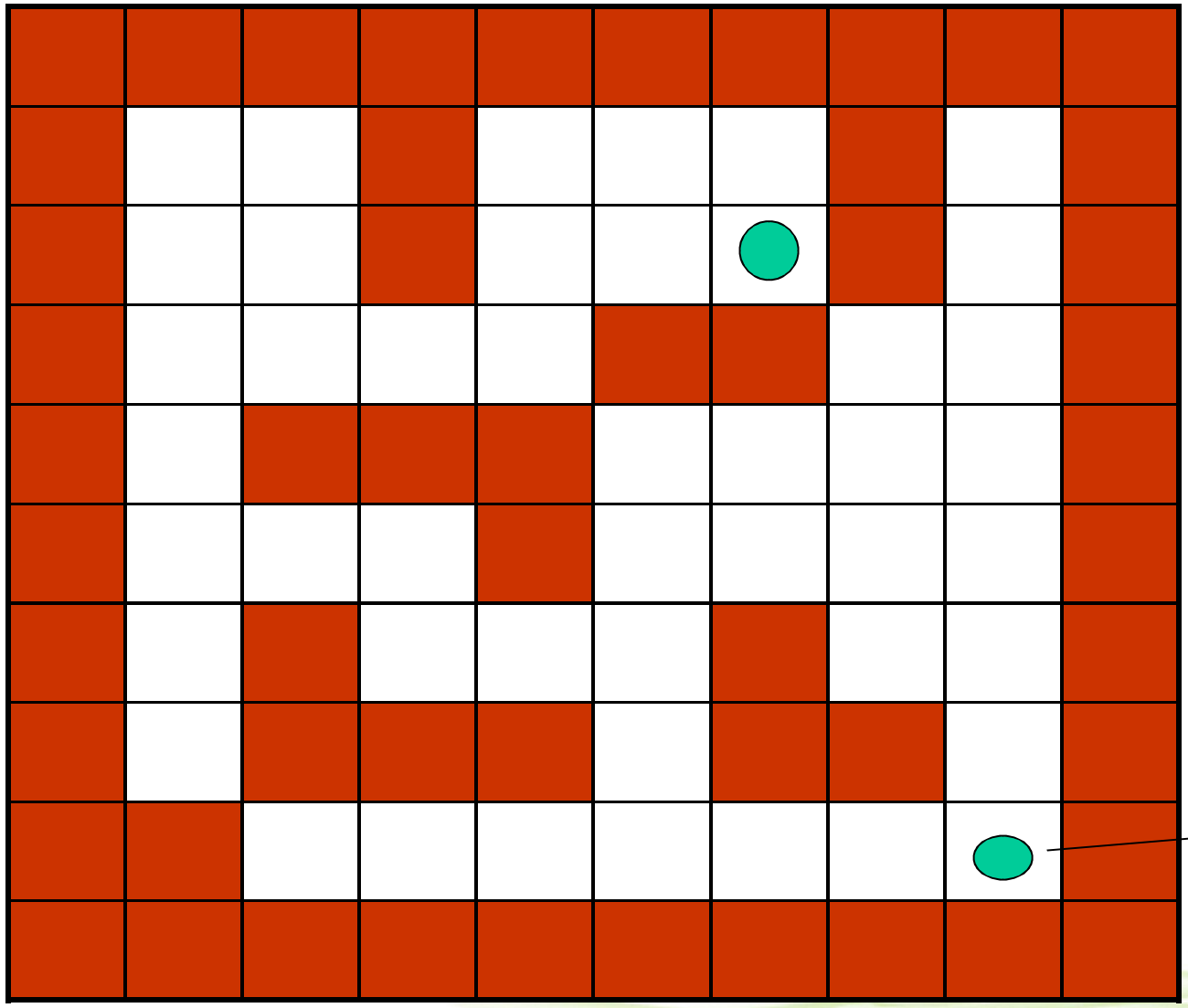
3.2.4 迷宫求解



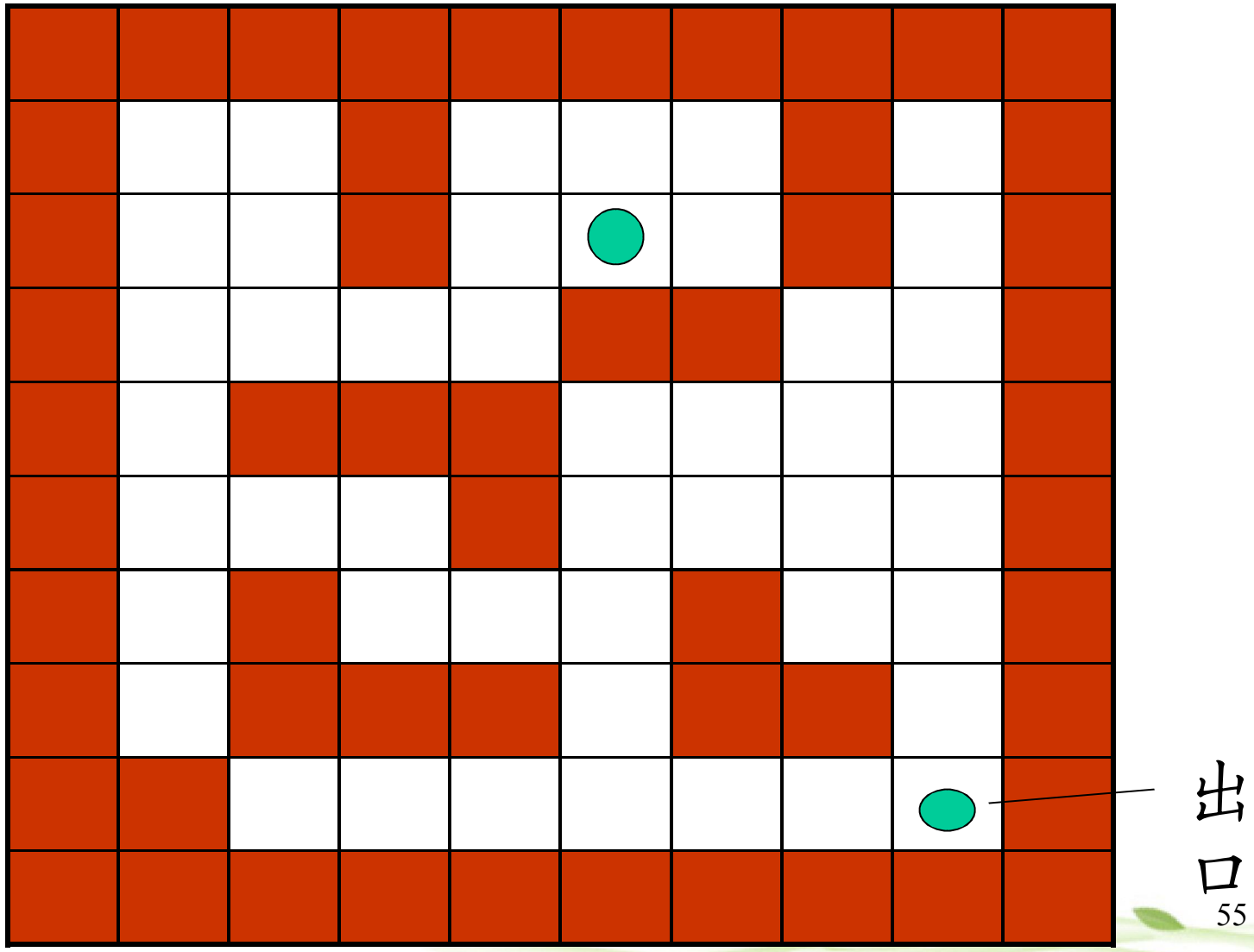
3.2.4 迷宫求解



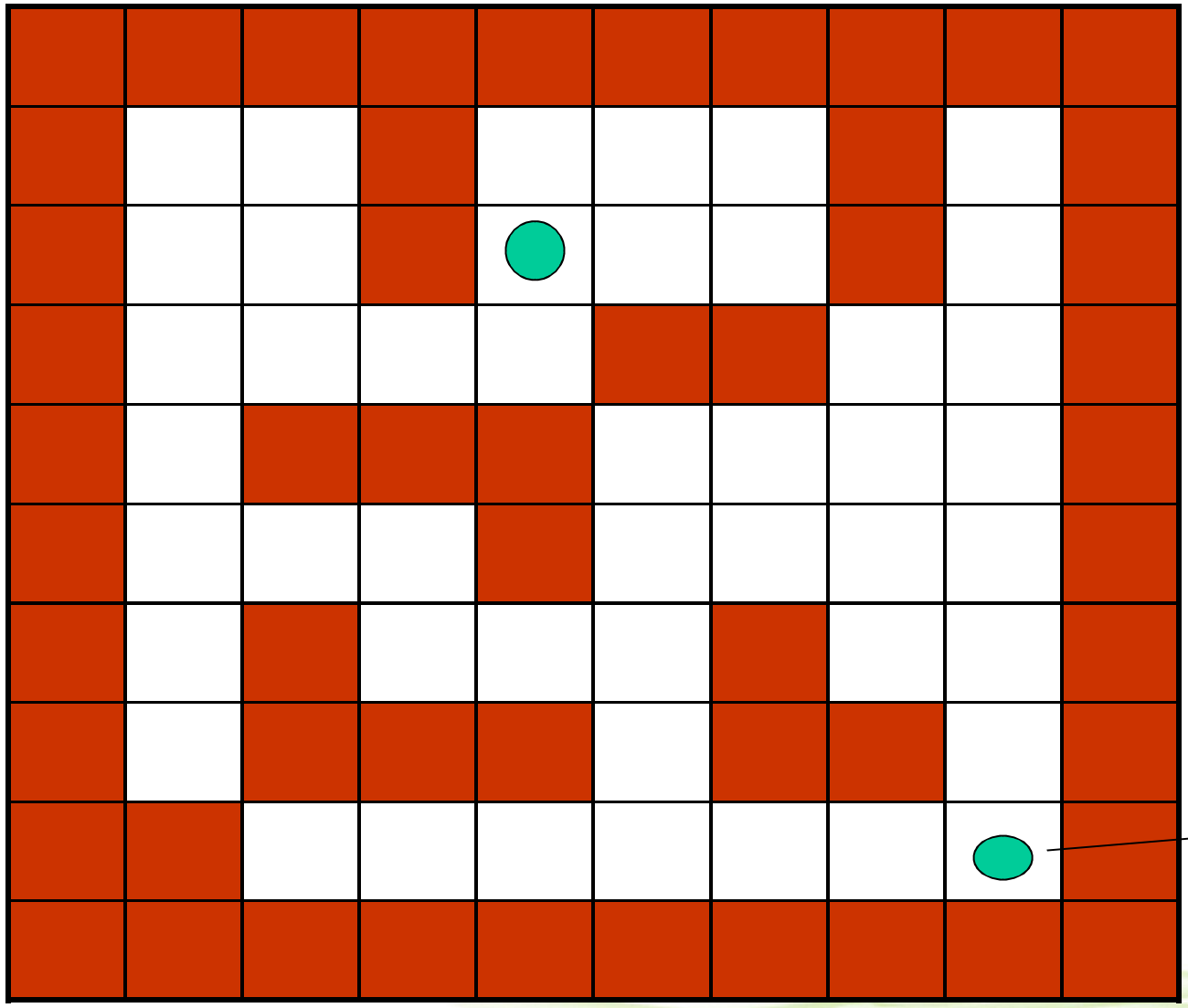
3.2.4 迷宫求解



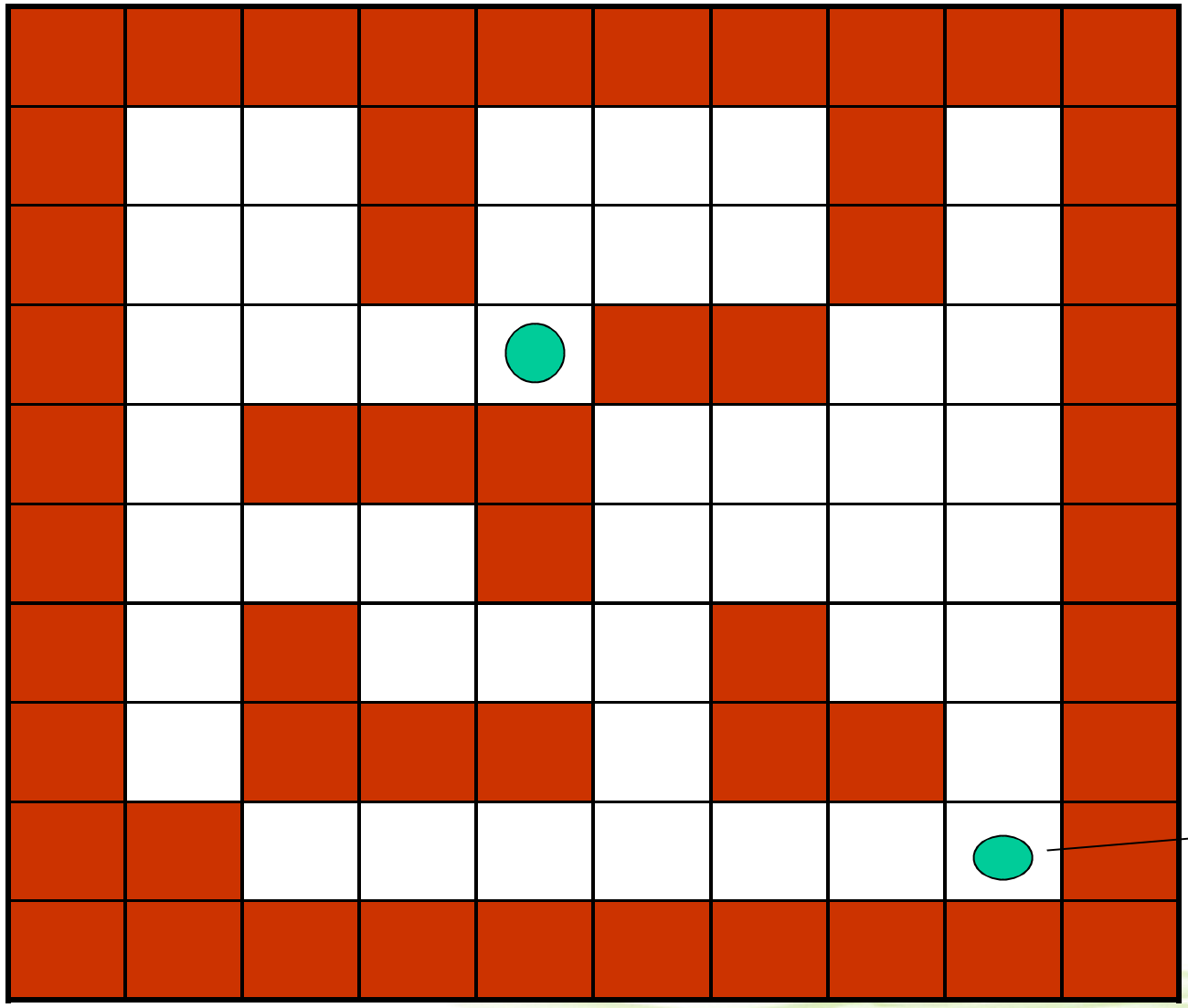
3.2.4 迷宫求解



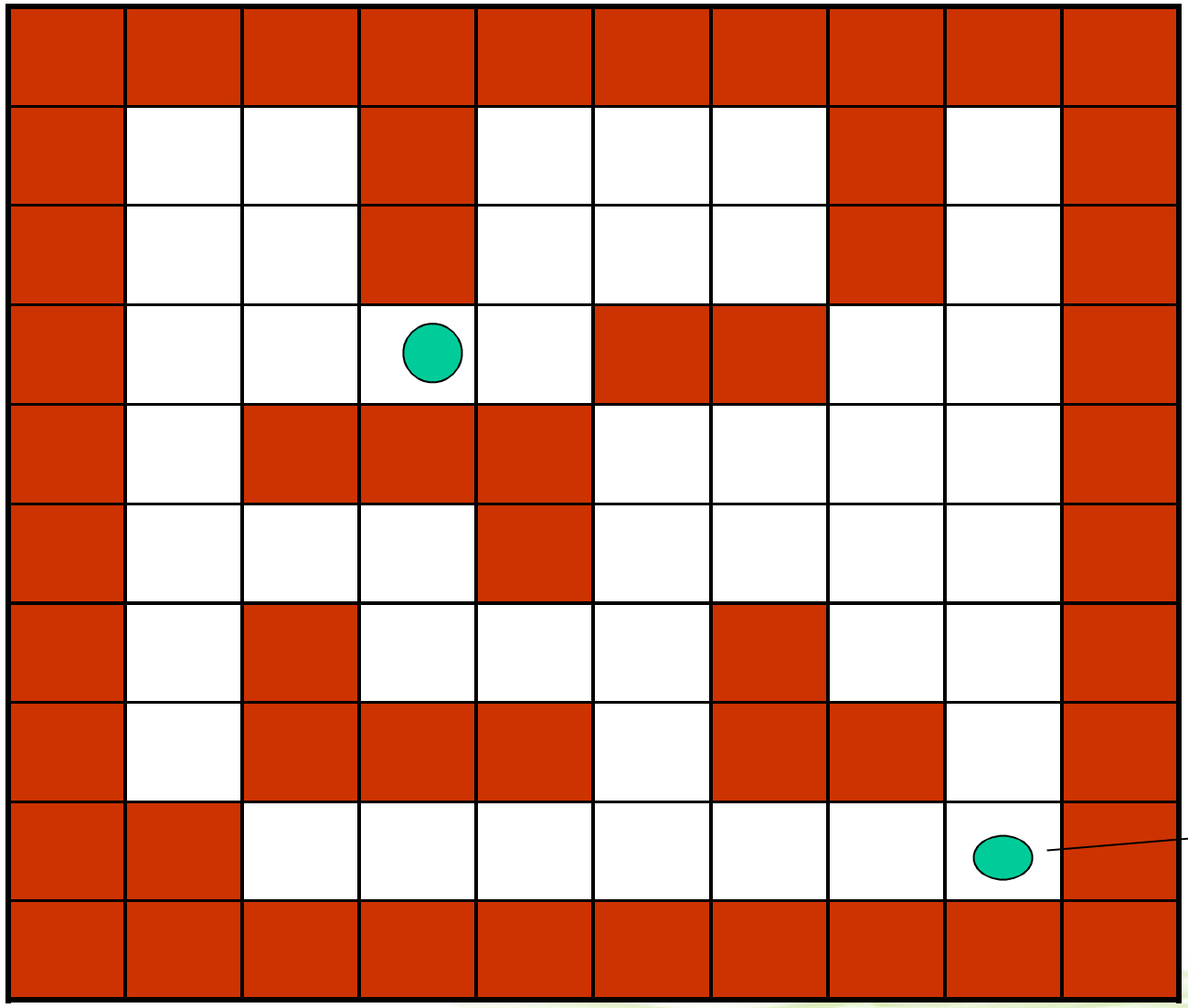
3.2.4 迷宫求解



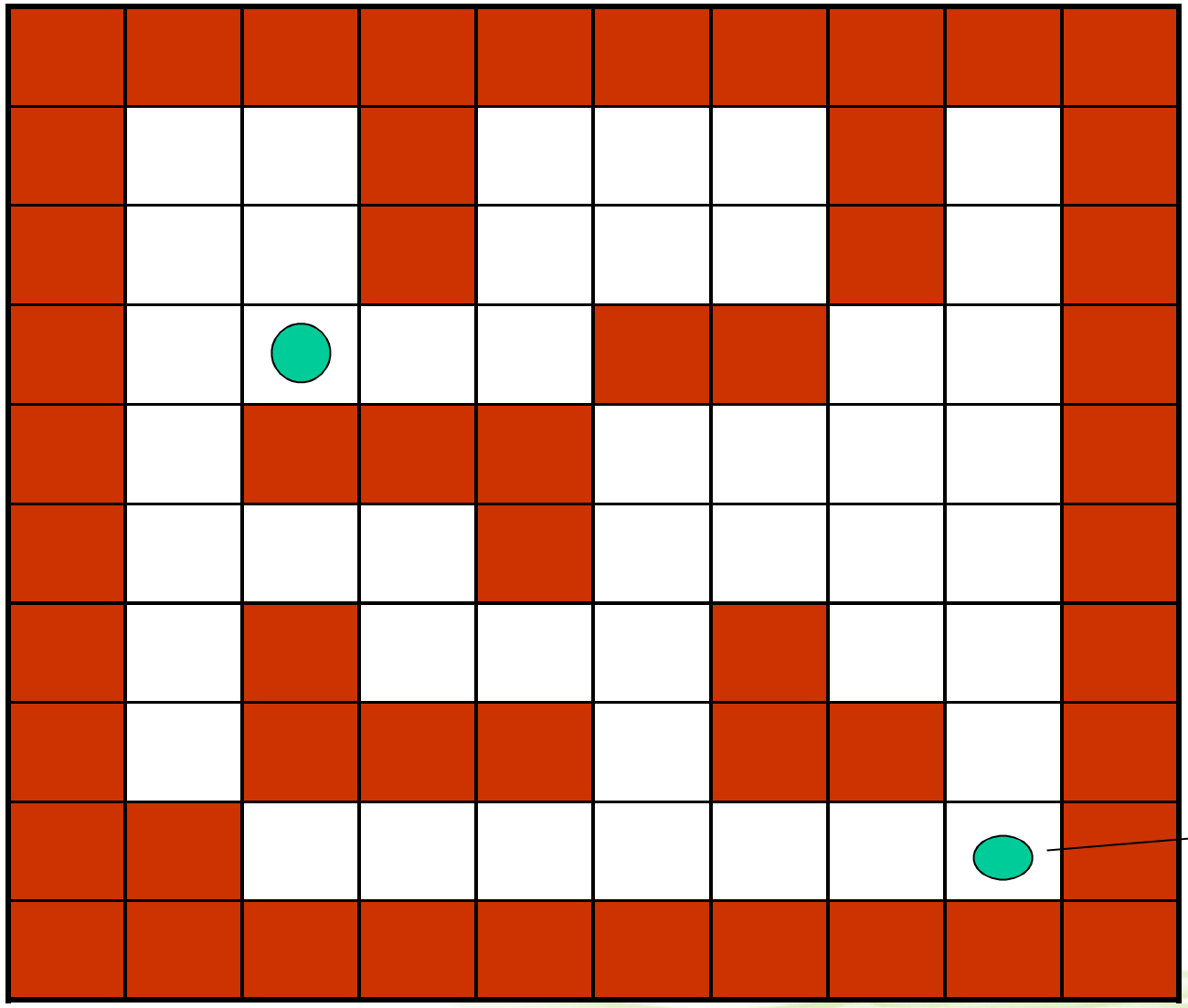
3.2.4 迷宫求解



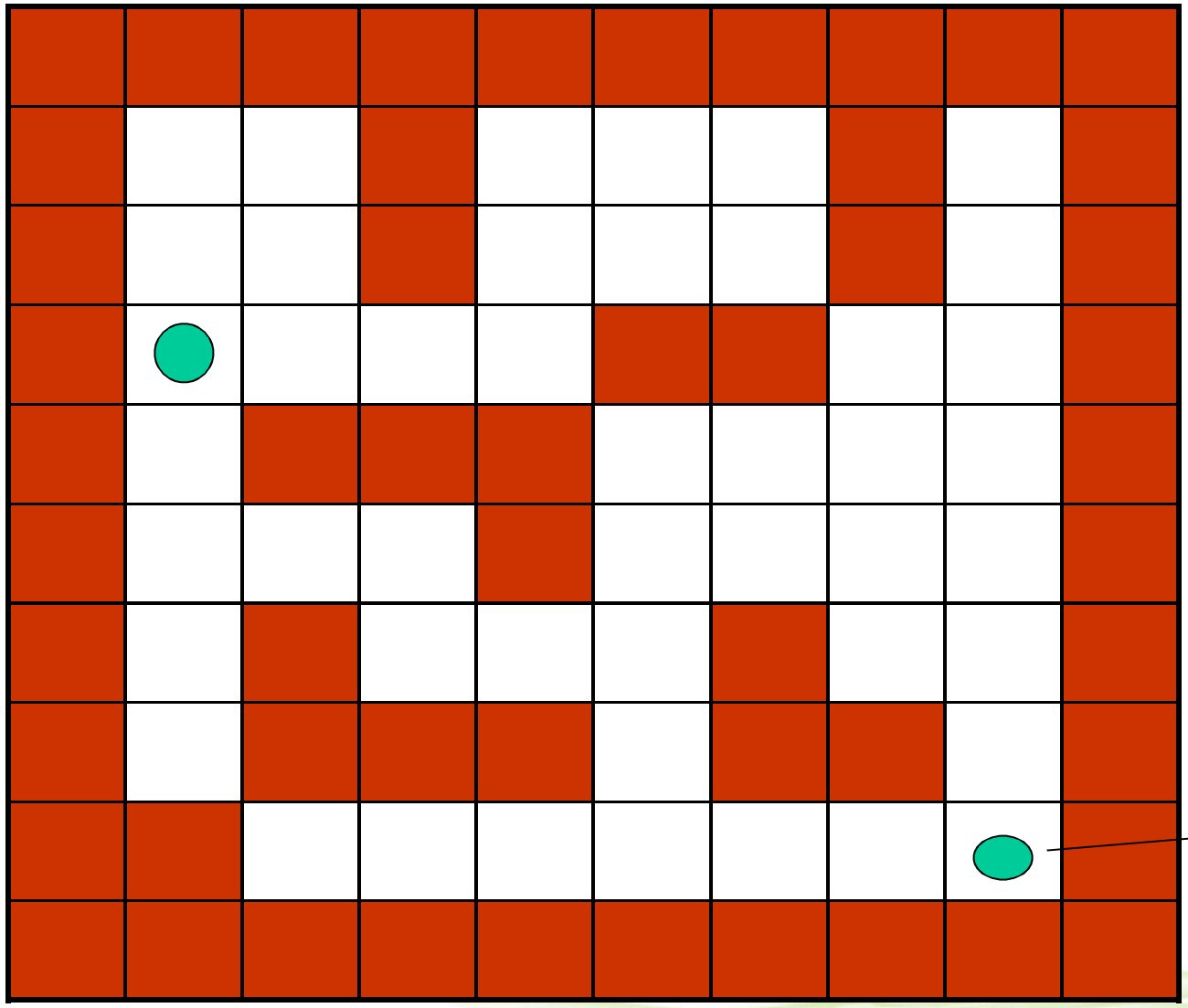
3.2.4 迷宫求解



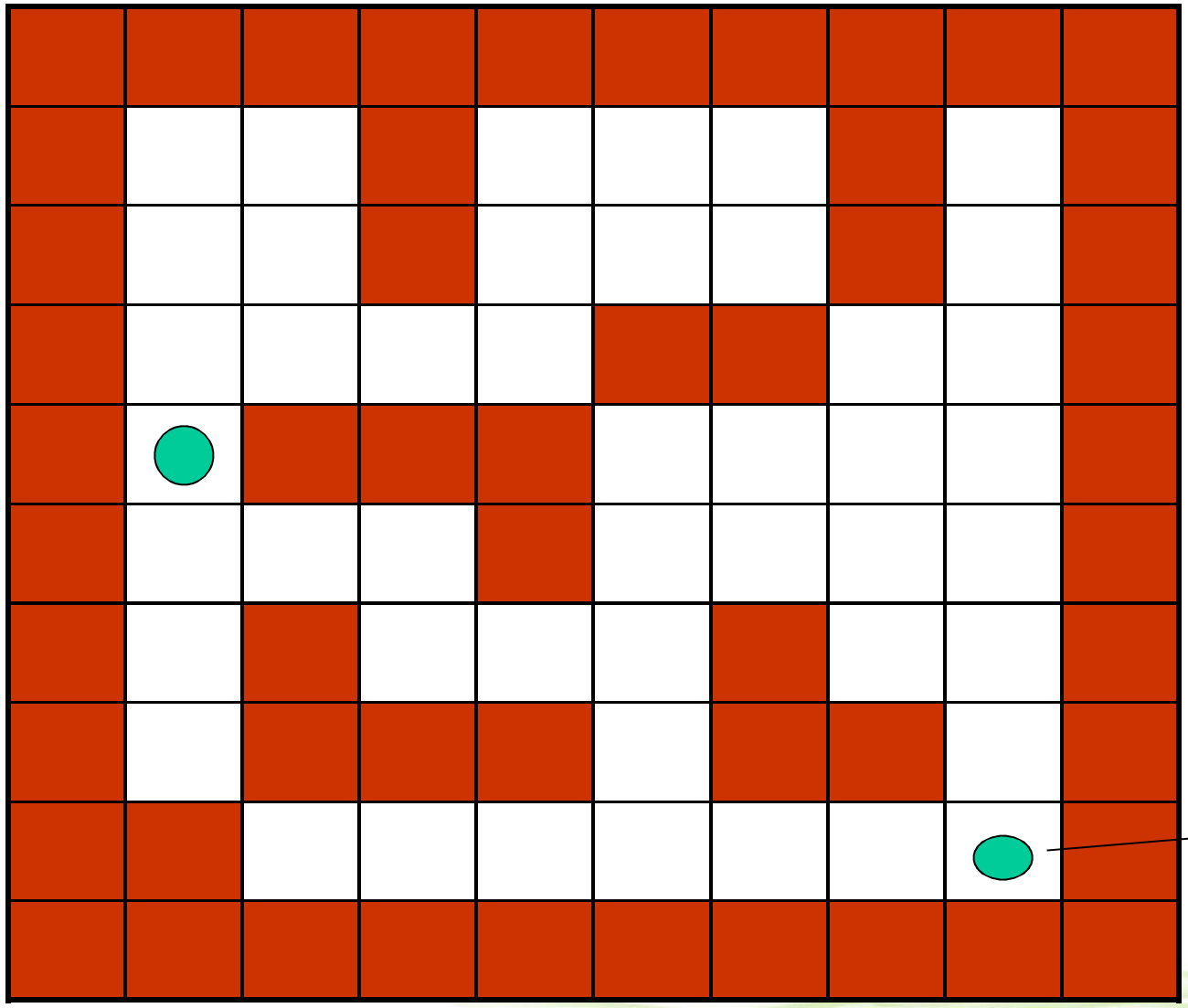
3.2.4 迷宫求解



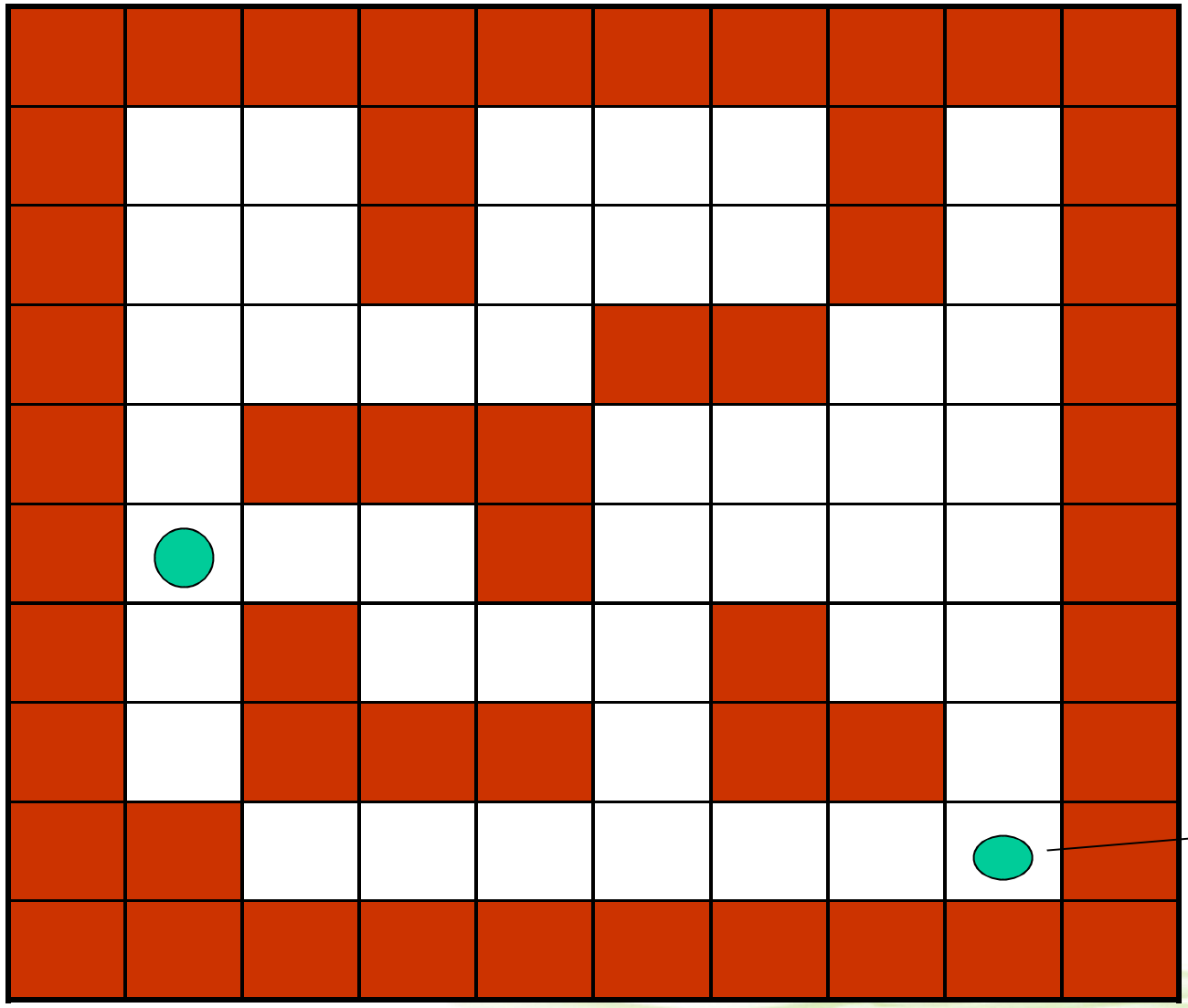
3.2.4 迷宫求解



3.2.4 迷宫求解



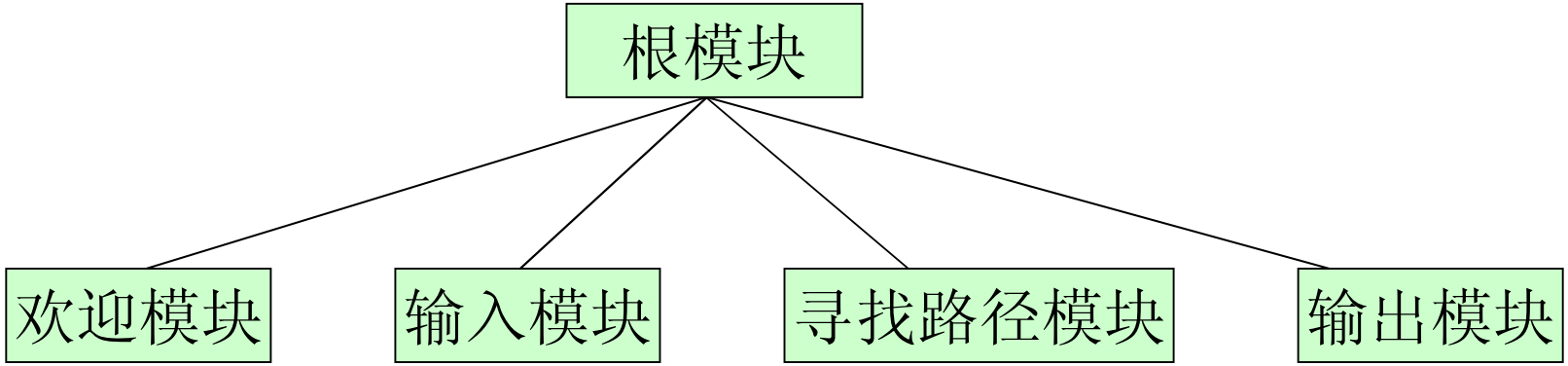
3.2.4 迷宫求解



求迷宫中一条从入口到出口的路径的算法：

```
设定当前位置的初值为入口位置;
 do {
  若当前位置可通,
  则{将当前位置插入栈顶;
       若该位置是出口位置，则算法结束;
       否则切换当前位置的东邻方块为新的当前位置;
  }
  否则,
       若栈不空且栈顶位置尚有其他方向未被探索,
          则设定新的当前位置为沿顺时针方向旋转找到的栈顶位置的下一相邻块;
       若栈不空但栈顶位置的四周均不可通,
          则{删去栈顶位置; //从路径中删去该通道块
               若栈不空，则重新测试新的栈顶位置,
                  直至找到一个可通的相邻块或出栈至栈空;
          }
} while (栈不空）;
```

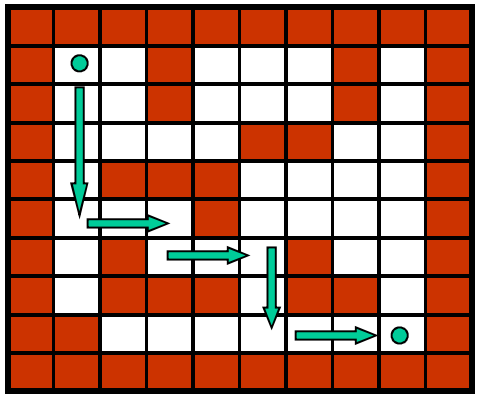
迷宫老鼠游戏软件的开发



Welcome To
Rat In A Maze

Square mazes only.
Please enter the number of rows.
Enter –1 to exit.
Press <Enter> when done.

FindPath()



时间复杂性分析 在最坏的情况下，可能要遍历每一个空闲的位置，而每个位置进入堆栈的机会最多有四次（或八次，在讨论8个移动方向时）。对于每个位置，需花费O(1)的时间来检查它的相邻位置。因此时间复杂度为O(空闲位置数目)。

Sartaj Sahni著. 数据结构、算法与应用----C++语言描述. 机械工业出版社，1999, p180.

第三章 栈和队列

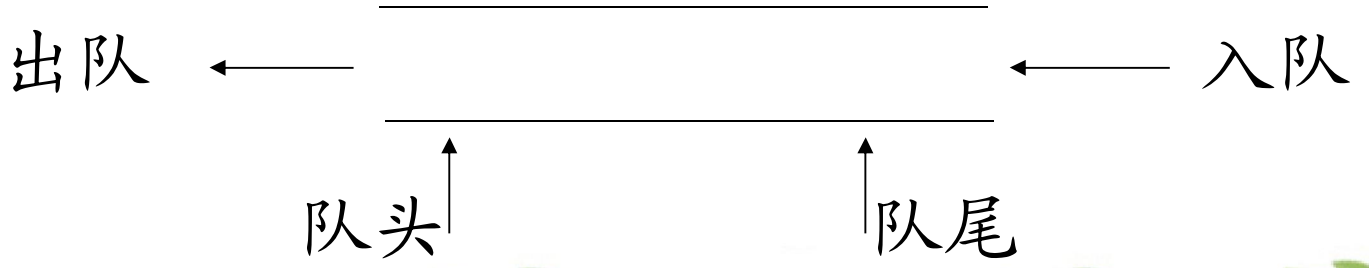
- 3.4 队列
  - 3.4.1 队列的ADT定义
  - 3.4.2 链队列
  - 3.4.3 循环队列

3.4 队列

队列(Queue)也是一种运算受限的线性表。它只允许在表的一端进行插入，而在另一端进行删除。允许删除的一端称为队头(front)，允许插入的一端称为队尾(rear)。

例如：排队购物。操作系统中的作业排队(打印)。先进入队列的成员总是先离开队列。因此队列亦称作先进先出(First In First Out)的线性表，简称FIFO表。

当队列中没有元素时称为空队列。在空队列中依次加入元素$a_1, a_2, \ldots a_n$之后，a_1是队头元素，a_n是队尾元素。显然退出队列的次序也只能是$a_1, a_2, \ldots a_n$ ，也就是说队列的修改是依先进先出的原则进行的。



3.4.1 队列的类型定义

ADT Queue {

数据对象:

$$D = \{a_i \mid a_i \in ElemSet,\ i=1,2,...,n,\ n \geq 0\}$$

数据关系:

$$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D,\ i=2,...,n\}$$

约定其中 a_1 端为队列头， a_n 端为队列尾

基本操作:

} **ADT** Queue

队列的基本操作：

InitQueue(&Q)　　**DestroyQueue(&Q)**

QueueEmpty(Q)　　**QueueLength(Q)**

GetHead(Q, &e)　　**ClearQueue(&Q)**

EnQueue(&Q, e)　　**DeQueue(&Q, &e)**

QueueTravers(Q, visit())

InitQueue(&Q)

操作结果：构造一个空队列Q。

DestroyQueue(&Q)

初始条件：队列Q已存在。

操作结果：队列Q被销毁，不再存在。

QueueEmpty(Q)

初始条件：队列Q已存在。

操作结果：若Q为空队列，则返回TRUE，否则返回FALSE。

QueueLength(Q)

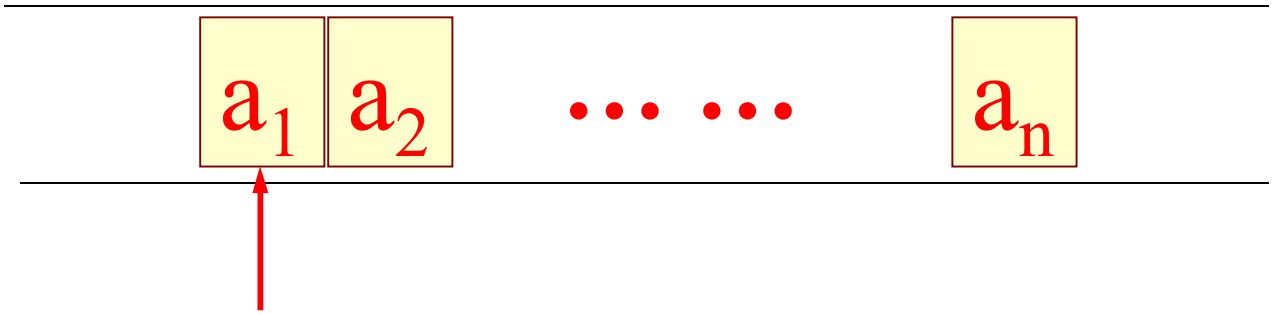
初始条件：队列Q已存在。

操作结果：返回Q的元素个数，即队列的长度。

GetHead(Q, &e)

初始条件：Q为非空队列。

操作结果：用e返回Q的队头元素。



ClearQueue(&Q)

初始条件：队列Q已存在。

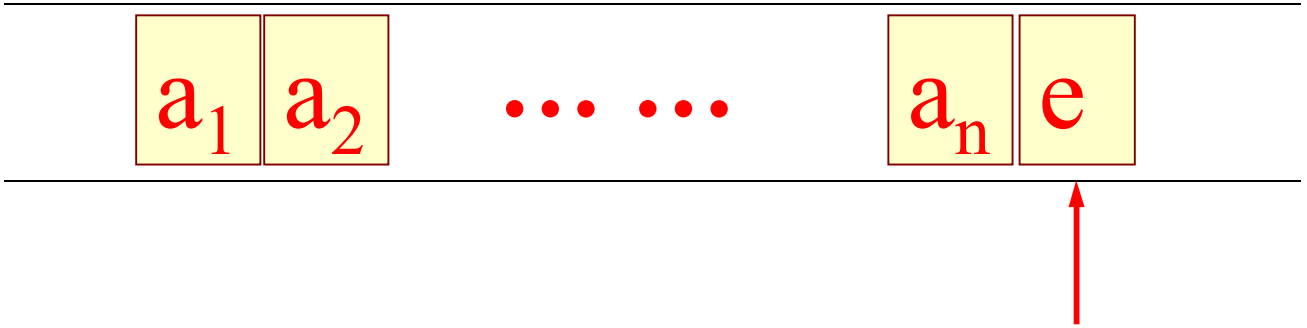
操作结果：将Q清为空队列。



EnQueue(&Q, e)

初始条件：队列Q已存在。

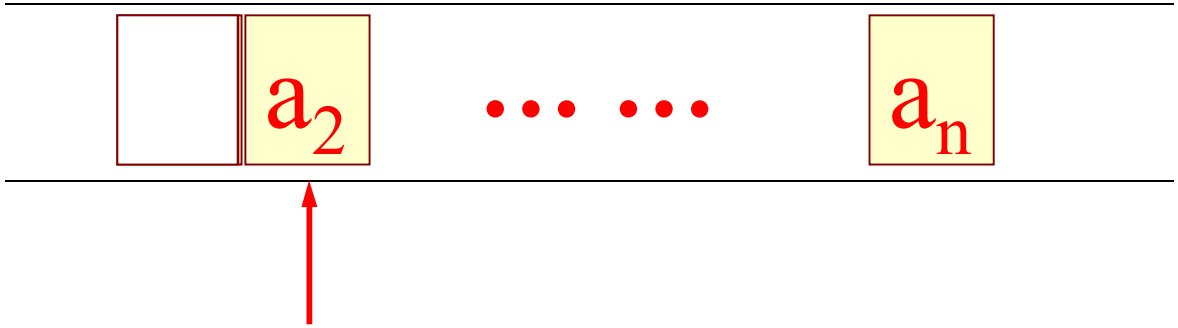
操作结果：插入元素e为Q的新的队尾元素。



DeQueue(&Q, &e)

初始条件：Q为非空队列。

操作结果：删除Q的队头元素，并用e返回其值。



QueueTravers(Q, visit())

初始条件：Q已存在且非空。

操作结果：从队头到队尾，依次对Q的每个数据元素调用函数**visit()**。一旦visit()失败，则操作失败。

3.4.2 队列类型的实现

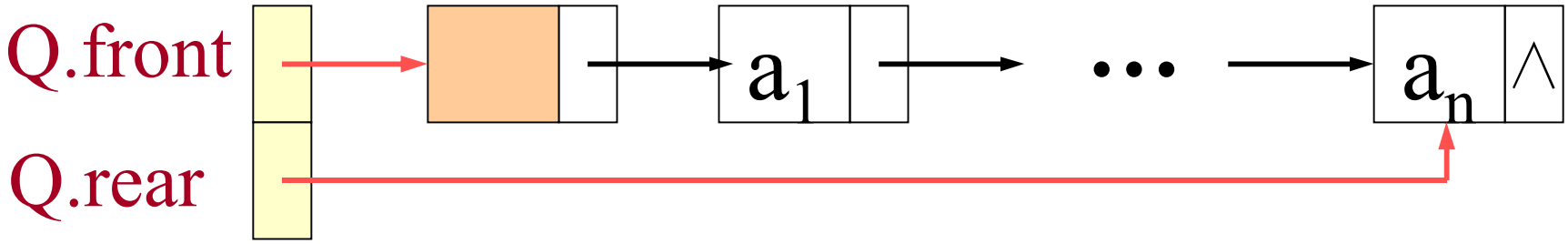
链队列——链式映象

循环队列——顺序映象

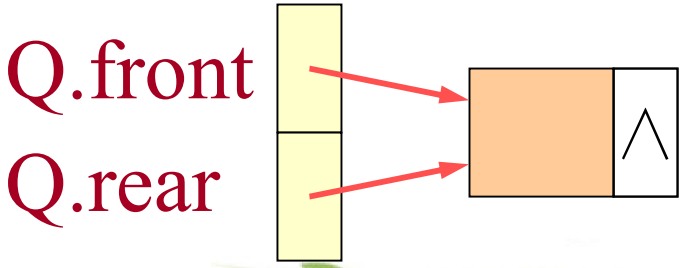
链队列——链式映象

```
typedef struct QNode {// 结点类型
  QElemType      data;
  struct QNode  *next;
} QNode, *QueuePtr;
```

```
typedef struct { // 链队列类型
    QueuePtr  front;  // 队头指针
    QueuePtr  rear;   // 队尾指针
} LinkQueue;
```

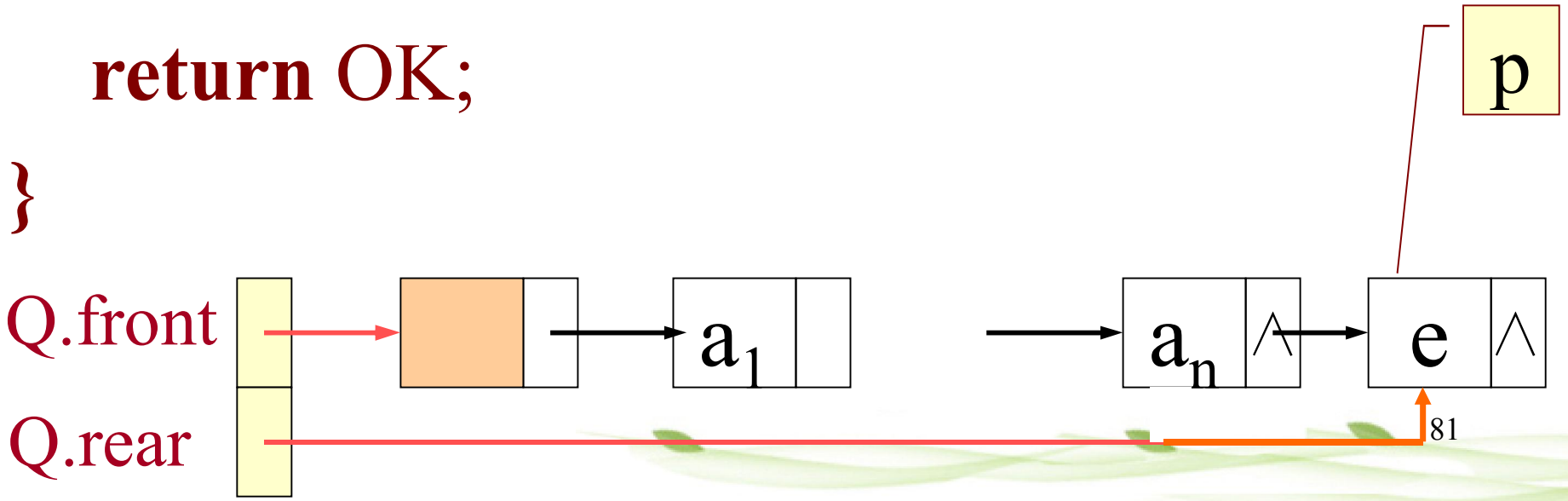


空队列



```
Status InitQueue (LinkQueue &Q) {
  // 构造一个空队列Q
  Q.front = Q.rear = new  QNode;
  if (!Q.front) exit (OVERFLOW); //存储分配失败
  Q.front->next = NULL;
  return OK;
}
```

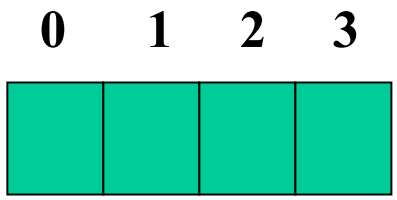
```
Status EnQueue (LinkQueue &Q, QElemType e) {
  // 插入元素e为Q的新的队尾元素
  p =  new QNode;
  if (!p)  exit (OVERFLOW);  //存储分配失败
  p->data = e;   p->next = NULL;
  Q.rear->next = p;    Q.rear = p;
  return OK;
}
```



```
Status DeQueue (LinkQueue &Q,
                                QElemType &e) {
 // 若队列不空，则删除Q的队头元素，
 //用 e 返回其值，并返回OK；否则返回ERROR
  if (Q.front == Q.rear)    return ERROR;
  p = Q.front->next;   e = p->data;
  Q.front->next = p->next;
   if (Q.rear == p)  Q.rear = Q.front;  队列被删空时
  delete (p);       return OK;
}
```

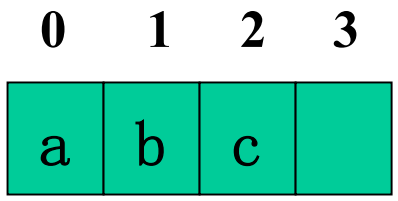
循环队列－顺序映象

队列的顺序存储结构称为顺序队列，用一个向量空间来存放当前队列中的元素。由于队列的队头和队尾的位置是变化的，因而要设两个指针分别指示队头和队尾元素在队列中的位置，它们的初始值在队列初始化时均应置为 0 。入队时将新元素插入所指的位置，然后加 1 。出队时，删去所指的元素，然后加 1 并返回被删元素。由此可见，当头尾指针相等时队列为空。在非空队列里，头指针始终指向队头(队列头元素)，而尾指针始终指向队尾元素的下一个位置。



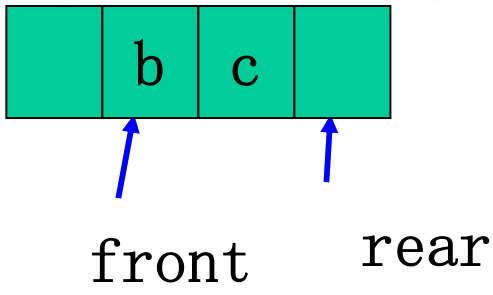
front=rear=0

(a)队列初始为空

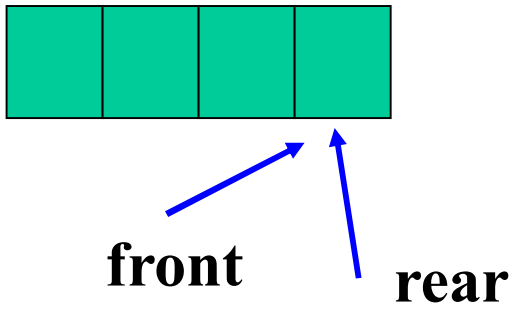


front=0 ,rear=3

（b）a, b, c入队



（ c）a出队



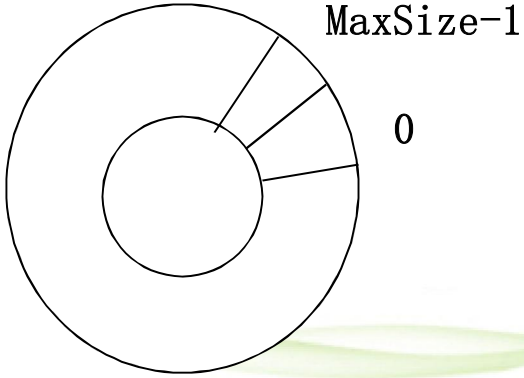
(d) b,c出队，队为空

为充分利用向量空间。克服上述现象，把向量空间想象为一个首尾相接的圆环，并称这种向量为循环向量，存储在其中的队列称为循环队列（Circular Queue）。在循环队列中进行出队、入队操作时，头尾指针仍要加1，朝前移动。只不过当头尾指针指向向量上界（MaxSize-1）时，其加1操作的结果是指向向量的下界0。

这种循环意义下的加1操作可以描述为：

```
if(i+1==MaxSize)
   i=0;
 else
   i++;
```

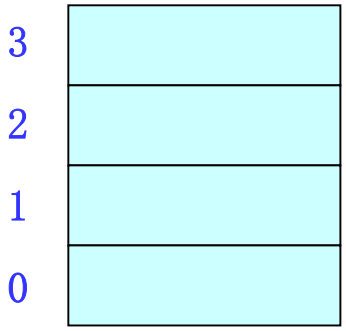
利用模运算可简化为：　i=(i+1)%MaxSize



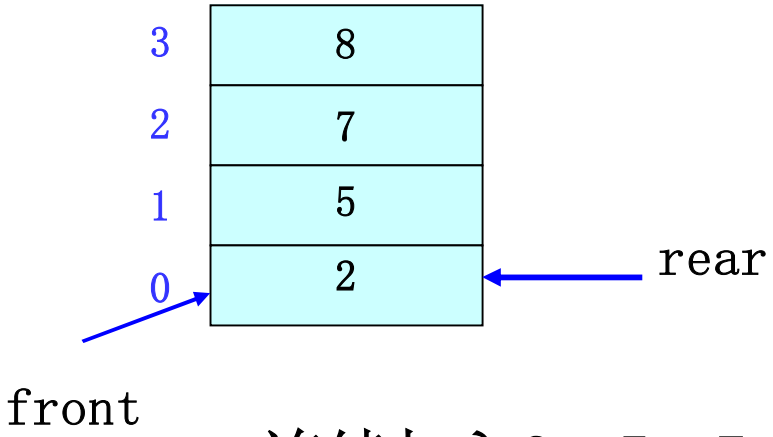
显然，因为循环队列元素的空间可以全被利用，除非向量空间真的被队列元素全部占用，否则不会上溢。因此，除一些简单的应用外，真正实用的顺序队列是循环队列。

入队时尾指针向前追赶头指针，出队时头指针向前追赶尾指针，故队空和队满时头尾指针均相等。因此，我们无法通过front==rear来判断队列“空”还是“满”。

开始：



front=rear=0



连续加入2，5，7，8后

front=rear=0

显然，因为循环队列元素的空间可以全被利用，除非向量空间真的被队列元素全部占用，否则不会上溢。因此，除一些简单的应用外，真正实用的顺序队列是循环队列。

入队时尾指针向前追赶头指针，出队时头指针向前追赶尾指针，故队空和队满时头尾指针均相等。因此，我们无法通过front=rear来判断队列“空”还是“满”。

解决此问题的方法至少有三种：

其一:另设一个布尔变量以区分队列的空和满；

其二:少用一个元素的空间，约定入队前，测试尾指针在循环意义下加1后是否等于头指针，若相等则认为队满；（但此时实际还有一个空位置）

其三 :是使用一个计数器记录队列中元素的总数（实际上是队列长度）。

```
Status InitQueue (SqQueue &Q) {
  // 构造一个空队列Q
  Q.base = (ElemType *) malloc
          (MAXQSIZE *sizeof (ElemType));
  if (!Q.base) exit (OVERFLOW);
                          // 存储分配失败
  Q.front = Q.rear = 0;
  return OK;
}
```

```
Status EnQueue (SqQueue &Q, ElemType e)
{  // 插入元素e为Q的新的队尾元素
   if ((Q.rear+1) % MAXQSIZE == Q.front)
      return ERROR; //队列满
   Q.base[Q.rear] = e;
   Q.rear = (Q.rear+1) % MAXQSIZE;
   return OK;
}
```

```
Status DeQueue (SqQueue &Q, ElemType &e)
{ // 若队列不空，则删除Q的队头元素，
  // 用e返回其值，并返回OK；否则返回ERROR
   if (Q.front == Q.rear)  return ERROR;
   e = Q.base[Q.front];
   Q.front = (Q.front+1) % MAXQSIZE;
   return OK;
}
```

本章小结

- 掌握栈和队列类型的特点，并能在相应的应用问题中正确选用它们。
- 熟练掌握栈类型的两种实现方法，特别应注意栈满和栈空的条件以及它们的描述方法。
- 熟练掌握循环队列和链队列的基本操作实现算法，特别注意队满和队空的描述方法。

作业3

3.1 设将整数1，2，3，4依次进栈，但只要出栈时栈非空，则可将出栈操作按任何次序夹入其中，请回答下述问题：

(1)若入、出栈次序为Push(1), Pop(),Push(2),Push(3), Pop(), Pop(), Push(4), Pop( ), 则出栈的数字序列为何(这里Push(i)表示i进栈，Pop( )表示出栈)?

(2) 能否得到出栈序列1423和1432? 并说明为什么不能得到或者如何得到。

3.2 设长度为n的链队列用单循环链表表示，若只设头指针，则怎样进行入队和出队操作；若只设尾指针呢？